

Compact Representations of Geometric Bipartite Graphs via Weighted Biclique Covers (Technical Report)

Anonymous Author(s)

Abstract

Bipartite graphs are a fundamental representation for relational data arising in recommendation systems, market-basket analysis, social networks, and communication graphs. A key challenge in these settings is to store and transmit large bipartite graphs compactly while preserving exact structural and path information. We study biclique-based representations of bipartite graphs $G = (V, U, E)$, where the edge set is encoded using a collection of complete bipartite subgraphs. We focus on the Weighted Biclique Covering problem, which minimizes the total number of vertices used across all bicliques, and introduce a generalized variant that additionally penalizes the number of bicliques, capturing practical overheads in storage, transmission, and model complexity.

While the weighted biclique covering problem is known to be NP-Complete, we show that the generalized variant is also NP-Complete. Despite this hardness, many real-world bipartite graphs admit low-dimensional geometric embeddings or can be well approximated by them. Leveraging this observation, we develop the first approximation algorithms with provable guarantees for the (generalized) weighted biclique covering problem on geometric bipartite graphs. Specifically, for δ -disk bipartite graphs (and intersection bipartite graphs) in low-dimensional ℓ_∞^d spaces, we design a polynomial-time algorithm that achieves an $O(\log |U| \cdot \log^d |V|)$ -approximation, combining ideas from greedy set cover, geometric range searching, and densest subgraph optimization. We also show how our algorithms extend to ℓ_α^d metrics for any $\alpha \geq 1$. Finally, we evaluate our algorithms on real-world bipartite datasets and show that they efficiently compute significantly smaller biclique-based representations than natural baselines, while scaling to large graphs.

1 Introduction

Bipartite graphs provide a natural and expressive framework for modeling relational data arising in a wide range of applications, including social and biological networks, job matching, dating platforms, recommendation systems, and resource allocation [61]. As modern datasets continue to grow in scale, the corresponding graphs often contain massive numbers of vertices and edges, making both storage and algorithmic processing increasingly challenging. In particular, the cost of running classical graph algorithms can become prohibitive as graph size and density increase.

Graph representation and compression techniques have therefore emerged as essential tools for mitigating these challenges. A common goal of such methods is to reduce the size of the graph, typically by decreasing the number of explicitly stored edges, while preserving its essential structural properties. For many downstream tasks, however, it is crucial that the compressed representation preserves path information, namely reachability and shortest-path relationships between vertices. When this information is retained,

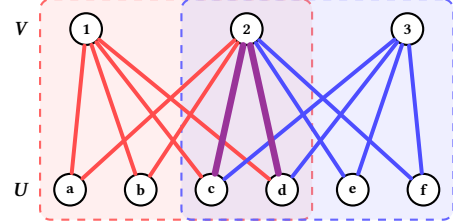


Figure 1: Example of graph compression using biclique edge covers. The red edges belong to the first biclique ($K_{2,4}$ defined by $\{1, 2\} \times \{a, b, c, d\}$), and the blue edges belong to the second biclique ($K_{2,4}$ defined by $\{2, 3\} \times \{c, d, e, f\}$). The purple edges belong to both bicliques. The total size of this cover is $6 + 6 = 12$. An optimal biclique partition (disallowing overlaps) is given by $\{1, 2\} \times \{a, b, c, d\}$, $\{2, 3\} \times \{e, f\}$, and $\{3\} \times \{c, d\}$, which results in a total size of $6 + 4 + 3 = 13$. The graph has 14 edges. Thus, $\text{Cover}(12) < \text{Partition}(13) < \text{Edges}(14)$.

the compressed graph can be used directly by path-sensitive algorithms such as matching, flow, and shortest-path computations, leading to substantial improvements in efficiency for large and dense graphs.

Motivating scenario. Consider a cloud computing environment consisting of multiple computing nodes connected by a network. A bipartite graph $G(V, U, E)$ is initially stored on a machine A, but an expensive downstream graph algorithm, such as shortest paths, matching, or flow computation, must be executed on a computing node B. A natural approach is to transfer the graph from A to B and run the algorithm on the explicit edge representation. However, transferring the entire graph requires sending at least $|E|$ objects over the network, and the subsequent running time of many classical graph algorithms also depends directly on the number of edges. For large and dense graphs, both the communication cost and the computational cost on B can therefore become prohibitively expensive. This motivates the need for a compact representation that can be transmitted more efficiently and, at the same time, supports faster execution of downstream graph algorithms by replacing the original graph with a smaller path-preserving representation.

A variety of graph compression and representation techniques [9, 12, 25, 26, 54] can be applied prior to transfer. However, most existing approaches incur substantial overhead when reconstructing complete path information, limiting their usefulness for downstream graph analytics. To address this limitation, prior work [27] and more recently [16] proposed a biclique-based representation that preserves exact path information. Specifically, the edge set of the bipartite graph G is *partitioned* into complete bipartite subgraphs (bicliques). For a biclique $S \subseteq G$ with vertex sets $V_S \subseteq V$ and $U_S \subseteq U$, the $|V_S| \cdot |U_S|$ edges are replaced by a single auxiliary node w_S connected to all vertices in $V_S \cup U_S$. This transformation replaces $|V_S| \cdot |U_S|$ edges by $|V_S| + |U_S|$ edges while preserving path

information in the resulting graph. The size of biclique S in the new representation is defined by $|V_S| + |U_S|$.

Limitations of biclique partitions. While biclique partitions can yield compact graph representations, this line of work suffers from two important limitations. First, the requirement that the bicliques form a *partition* of the edge set is overly restrictive. Under this requirement, each edge of G must belong to exactly one biclique, severely limiting the flexibility of the representation. Second, prior work does not explicitly study the optimization problem of constructing a representation of minimum size.

In contrast, we observe that requiring only a *cover* of the edges by bicliques (allowing edges to appear in multiple bicliques) is sufficient to preserve path information using the same node-replacement construction. Furthermore, covering the edges by bicliques always leads to strictly smaller representations because every partition is a cover, however not every cover is a partition. Consequently, allowing coverings can only improve the quality of the resulting representation. Figure 1 presents an example illustrating that a biclique cover yields a smaller representation.

Weighted biclique covering. Motivated by these observations, we study the problem of covering the edges of a bipartite graph G with bicliques so as to minimize the size of the resulting representation, which is known as the *Weighted Biclique Cover* problem (WBC) [56]. Prior work on WBC has primarily only focused on upper and lower bounds on the optimal solution size [19, 23, 56, 57]. The problem has also been studied in the role-mining literature under the name *edge-RMP* [58], where it is shown to be NP-complete. Existing approaches for WBC (and equivalently edge-RMP) [46, 48, 58] rely on heuristics and do not provide theoretical guarantees.

A generalized cost model. While we provide the first approximation algorithms for the WBC problem on certain classes of graphs, Minimizing only the total size of the selected bicliques does not fully capture several realistic deployment scenarios. In many practical settings, an additional fixed cost is incurred for each selected biclique, independent of its size. Returning to the cloud computing scenario, suppose that the edges of each biclique are transferred as a single packet. Each packet incurs a fixed overhead that is independent of its payload (e.g., a network header). Let c denote this per-biclique overhead. To better capture such costs, we introduce a generalized objective that accounts for both the total size of the selected bicliques and their number. We formalize this extension as the *Generalized Weighted Biclique Cover* problem (GWBC), whose goal is to minimize the sum of the sizes of the selected bicliques plus c times their number. Although the GWBC problem has not been explicitly defined in prior work, it naturally interpolates between two known problems. When $c = 0$, GWBC reduces to the WBC problem. In contrast, for sufficiently large values of c , GWBC reduces to the *bipartite dimension problem*, which seeks to minimize the number of bicliques needed to cover all edges of the input graph. While inapproximability results and exact algorithms for specific graph classes are known for the bipartite dimension problem on specific graph classes [4, 13, 14, 28, 35, 40, 44, 47, 55, 66], these results do not extend to GWBC.

Geometric structure and our approach. Despite the long history of WBC and its relevance to data compression and distributed

computation, no approximation algorithms with theoretical guarantees are known, either for WBC or for its generalized variant GWBC. In this paper, we build on the observation that many graphs arising in practice are naturally *disk graphs* [8, 41, 43] (also referred to as *proximity graphs* [1]) or *intersection graphs* [37–39] or can be well approximated as such. In these graphs, nodes are embedded as points in a geometric space, and edges connect pairs of nodes that are sufficiently close. For example, social networks and co-authorship graphs can often be embedded, with small error, into the space of user profiles with low intrinsic dimension [60, 62]. More generally, for many networks across diverse domains (e.g., social, transportation, Internet), suitable embeddings exist that approximately preserve structural properties and shortest paths of the original graphs [60, 64, 65]. Although these graphs can be represented implicitly via their geometric embeddings, most classical graph algorithms require an explicit edge-based representation. Consequently, even geometrically embedded graphs must often be materialized and transmitted as conventional graphs.

In this paper, we leverage the geometric structure of bipartite disk and intersection graphs to design the first approximation algorithms with provable guarantees for both WBC and GWBC.

1.1 Problem Definitions and our Contributions

Let $G(V, U, E)$ be a bipartite graph with two sets of vertices V and U , with $n = |V| + |U|$, and a set of edges $E \subseteq V \times U$, with $m = |E|$. For a subgraph $G' \subseteq G$, let $\mathcal{V}(G')$ (or $\mathcal{U}(G')$) be the vertices of V (or U) that are included in G' . We also define $\mathcal{D}(G') = \mathcal{V}(G') \cup \mathcal{U}(G')$, and $|\mathcal{D}(G')| = |\mathcal{V}(G')| + |\mathcal{U}(G')|$. Let $\mathcal{E}(G')$ be the set of edges in the subgraph G' .

Definition 1. (*Biclique*) A subgraph $S \subseteq G$ of a given bipartite graph G is a *biclique* (also called *complete bipartite subgraph*) if for every pair $v \in \mathcal{V}(S)$ and $u \in \mathcal{U}(S)$, $(v, u) \in \mathcal{E}(S)$.

Definition 2. (*Biclique Edge Cover*) A collection of subgraphs $S = \{S_1, \dots, S_k\}$, is called a *biclique edge covering* of a given graph G , if for every $i = 1, \dots, k$, S_i is a biclique of G , and for each $e \in \mathcal{E}(G)$ there exists at least one $S_j \in S$, such that $e \in \mathcal{E}(S_j)$.

Next, we define two optimization problems studied in this paper.

Definition 3. (*Weighted Biclique Covering (WBC)*) Given a bipartite graph $G(V, U, E)$ the goal is to construct a biclique edge cover $S = \{S_1, \dots, S_k\}$ for G , with minimum cost $\mu(S) = \sum_{S_i \in S} |\mathcal{D}(S_i)|$.

Definition 4. (*Generalized Weighted Biclique Covering (GWBC)*) Given a bipartite graph G and a parameter $c \geq 0$ the goal is to construct a biclique edge cover $S = \{S_1, \dots, S_k\}$ for G , with minimum cost $\sigma(S) = c \cdot k + \sum_{S_i \in S} |\mathcal{D}(S_i)|$.

In order to study the hardness we also define the decision version of the problems: Given a parameter $\tau > 0$ the goal is to decide whether there exists a biclique edge cover S such that $\mu(S) \leq \tau$ for the WBC problem and $\sigma(S) \leq \tau$ for the GWBC problem. We refer to these problems as dWBC and dGWBC, respectively.

For a real number $\beta > 1$, an algorithm is a β -approximation algorithm for the GWBC (resp. WBC) problem if it returns a biclique edge cover S such that $\sigma(S) \leq \beta \cdot \sigma(S^*)$ (resp. $\mu(S) \leq \beta \cdot \mu(S^*)$), where S^* is the biclique edge cover with the minimum cost.

For any $\alpha \geq 1$, let $\psi_\alpha : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ be the distance function in ℓ_α^d metric, defined as follows. For every pair of points $v, u \in \mathbb{R}^d$,

$\psi_\alpha(v, u) = \left(\sum_{j=\{1, \dots, d\}} (v(j) - u(j))^\alpha \right)^{1/\alpha}$ where $v(j)$ (resp. $u(j)$) is the j -th coordinate of point v (resp. u). For $\alpha = \infty$ we have, $\psi(v, u) = \max_{j=\{1, \dots, d\}} |v_j - u_j|$.

Next, we formally define the δ -disk bipartite graph and the intersection bipartite graph.

Definition 5. (δ -disk bipartite graph) A bipartite graph $G(V, U, E)$ is a δ -disk bipartite graph in ℓ_α^d metric if V and U are two finite sets of points in $\mathbb{R}^d$, and for any pair of points $v \in V$ and $u \in U$ there exists an edge $(v, u) \in E$ if and only if $\psi_\alpha(v, u) \leq \delta$.

A hyper-rectangle r in $\mathbb{R}^d$ is defined as $r = \times_{j=1}^d I_j$, where each I_j is an interval specifying the range along the j -th coordinate. A hyper-rectangle is called a *box* if all its side lengths are equal, that is, if all intervals I_j have the same length.

Definition 6. (Intersection bipartite graph) A graph $G(V, U, E, \mathcal{R}_V, \mathcal{R}_U)$ is an intersection bipartite graph in $\mathbb{R}^d$ if every node $v \in V$ ($u \in U$) is associated with hyper-rectangle $r_v \in \mathcal{R}_V$ ($r_u \in \mathcal{R}_U$) in $\mathbb{R}^d$, and for any pair of points $v \in V$ and $u \in U$ there exists an edge $(v, u) \in E$ if and only if $r_v \cap r_u \neq \emptyset$.

Our Contributions. Our results are summarized as follows.

- In Section 2, we show that dGWBC is NP-complete for any integer cost c via a reduction from the dWBC problem.

- To the best of our knowledge, no approximation algorithm with provable guarantees have previously been studied for the WBC problem, nor for its generalization GWBC, either on general graphs or on restricted graph classes. In Section 3, we present the first polynomial time $O(\log^d n)$ -approximation algorithm for the GWBC (and WBC) problem on δ -disk graphs in the ℓ_∞^d metric.

- In Section 4, we extend our algorithmic framework to the more general class of intersection bipartite graphs. This extension yields also the first approximation algorithms for the WBC and GWBC problems on interval graphs and k -nearest-neighbor bipartite graphs. Moreover, we discuss how our techniques for the WBC and GWBC problems on δ -disk graphs generalize to the ℓ_α^d metric for any $\alpha \geq 1$. In particular, we present a practical $O(\lambda \log n)$ -approximation algorithm that applies to any ℓ_α^d metric, where λ is the range cover size parameter defined in Subsection 4.2.1.

- We implement our practical algorithm for δ -disk graphs under both the ℓ_∞^d and ℓ_2^d metrics, for both WBC and GWBC. In Section 5, we evaluate our methods on real-world datasets and show that they consistently outperform all baselines, producing (generalized) weighted biclique covers that are significantly smaller (often by up to a factor of four) while achieving comparable running times and substantially lower memory usage.

2 Hardness of GWBC

It is known that covering the edges of a bipartite graph with the minimum number of bicliques (the bipartite dimension problem) is NP-complete [51]. Similarly, minimizing the total size of the bicliques in a cover (the dWBC problem) is also NP-complete [58]. The GWBC problem generalizes both objectives by jointly accounting for the number and total size of the selected bicliques. Indeed, when $c = 0$, GWBC reduces to WBC, while for sufficiently large c (e.g., $c \geq n^6$), it reduces to the bipartite dimension problem. Since both extremes are NP-complete, we show that for any fixed integer value of c , the decision version dGWBC is also NP-complete.

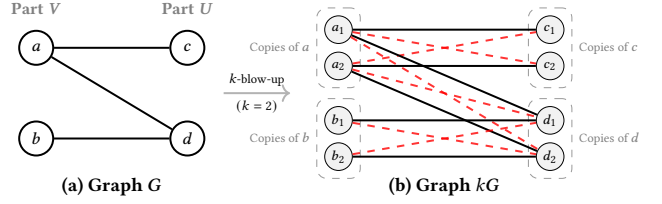


Figure 2: Illustration of the blow-up graph construction with $k = 2$. (a) A bipartite graph G with vertices $V = \{a, b\}$ and $U = \{c, d\}$. (b) The constructed graph kG . Solid black edges connect corresponding copies, while dashed red edges connect non-corresponding copies.

To prove the hardness of dGWBC, we present a polynomial-time reduction from the dWBC problem. Let $G(V, U, E)$ be a bipartite graph and let τ be a positive integer representing the target maximum weight for the dWBC problem over G . We construct a corresponding graph and target cost for the dGWBC problem as follows. Let $m = |E|$ be the number of edges in G . We choose an integer k such that $k > cm$. We define the graph G' to be a k -blow-up of G , denoted as kG . For every vertex $x \in \mathcal{D}(G)$, we create k copies in kG , denoted by $\{x_1, x_2, \dots, x_k\}$. If $\{u, v\} \in E$, we connect every copy of u to every copy of v in kG . Thus, if $u \in V$, $v \in U$, and $\{u, v\} \in E$, then for all $i, j \in [k]$, there is an edge between u_i and v_j in kG . More formally, we set $\mathcal{D}(kG) = \{x_i \mid x \in \mathcal{D}(G), i \in [k]\}$ and $\mathcal{E}(kG) = \{(u_i, v_j) \mid u, v \in \mathcal{D}(G), i, j \in [k]\}$. An example of this construction is shown in Figure 2.

We set the target cost for the dGWBC problem on kG to be $\tau' = k\tau + cm$. We now show that a solution to WBC on G with cost at most τ corresponds to a solution to GWBC on kG with cost at most τ' , and vice versa. The proof of the next lemma is shown in Appendix A.

Lemma 2.1. *There exists a biclique edge cover S for G such that $\mu(S) \leq \tau$, if and only if there exists a biclique edge cover S' for kG such that $\sigma(S') \leq k\tau + cm$.*

THEOREM 2.2. *For any fixed integer $c \geq 0$, the dGWBC problem is NP-complete.*

PROOF. First, we observe that dGWBC is in NP. Given a collection of subgraphs, we can verify in polynomial time whether each subgraph is a biclique, whether they cover all edges in E , and whether the total cost σ is at most the given threshold.

Next, we show NP-hardness. If $c > n^6$, the cost term $c \cdot m$ dominates the weight term $\sum_{S_i \in S} |\mathcal{D}(S_i)|$. In this case, minimizing σ becomes equivalent to minimizing the number of bicliques, which is the Bipartite Dimension problem, known to be NP-complete [51]. Therefore, we can assume $c \leq n^6$, so we use the reduction described above. We choose $k = cm + 1$ (so that we have $k > cm$). Since $m \leq n^2$ and $c \leq n^6$, we have $k \leq n^8$, and hence k is bounded by a polynomial in n . Consequently, the constructed graph kG , which has $k \cdot n$ vertices, has size polynomial in the size of the input graph G . By Lemma 2.1, the instance of dWBC has a solution with cost at most τ if and only if the instance of dGWBC has a solution with cost at most $k\tau + cm$. $\square$

3 Approximation Algorithms

We design an approximation algorithm for the GWBC problem for any $c \geq 0$ in δ -disk bipartite graphs in ℓ_∞^d metric, where $d = O(1)$.

Since our algorithm also works for $c = 0$, it is also valid for the WBC problem. In the ℓ_∞^d metric a ball with center a point $u \in \mathbb{R}^d$ and radius δ is a box ρ_u in $\mathbb{R}^d$ with center u and equal size side lengths 2δ . It is straightforward that if G is a δ -disk bipartite graph then for every $v \in V$ and $u \in U$, $(v, u) \in E$ if and only if $v \in \rho_u$. Throughout the section, we consider that $n_V = |V|$, $n_U = |U|$, $n = n_V + n_U$, $m = |E|$, and without loss of generality $n_V \leq n_U$.

High level ideas. Before we proceed with the technical details we give the high level ideas of our algorithm. We first focus on WBC and then extend to GWBC. We start by reducing WBC to an instance of the weighted set cover problem. Each edge corresponds to an element, and each biclique to a set including its edges, with weight equal to the number of nodes in the biclique. We apply the classical greedy $O(\log n) = O(\log n_U)$ -approximation algorithm for weighted set cover, which also yields an $O(\log n_U)$ -approximation for WBC. Let Δ denote the set of uncovered edges at the beginning of an iteration of the greedy algorithm. The algorithm selects the biclique S^* that minimizes $\frac{|\mathcal{D}(S^*)|}{|\mathcal{E}(S^*) \cap \Delta|}$. However, a fundamental obstacle arises: the number of bicliques in a graph may be exponential in the number of nodes. As a result, a naive implementation of the greedy algorithm is computationally expensive.

A key observation is that the greedy algorithm does not require selecting the optimal biclique in each iteration. It suffices to find a biclique S such that $\frac{|\mathcal{D}(S)|}{|\mathcal{E}(S) \cap \Delta|} \leq \hat{\gamma} \cdot \frac{|\mathcal{D}(S^*)|}{|\mathcal{E}(S^*) \cap \Delta|}$. In this case, the greedy algorithm yields an $O(\hat{\gamma} \cdot \log n_U)$ -approximation.

A natural first attempt is to compute a *densest subgraph* of G restricted to uncovered edges. The densest subgraph problem finds a subgraph that minimizes the ratio of the number of edges over the number of nodes in polynomial time [31]. Although this objective matches the greedy criterion for our algorithm, the resulting subgraph is not necessarily a biclique. To address this, we exploit the geometric structure of δ -disk graphs using a range tree [24] built over V . A range tree creates $O(n_V \log^{d-1} n_V)$ hyper-rectangles C , such that for any hyper-rectangle ρ in $\mathbb{R}^d$ there are always $O(\log^d n)$ non-intersecting hyper-rectangles in C that contains exactly the points in $\rho \cap V$.

Using this property we show that there should exist a hyper-rectangle $x \in C$ and a subgraph $G_x(V_x, U_x, E_x)$ such that i) $V_x \subseteq V$, ii) U_x is the set of points in U within distance δ from all points in V_x , iii) E_x is the set of uncovered edges between V_x and U_x , and iv) the densest subgraph of G_x contains at least $\frac{1}{O(\log^d n)} \cdot |\mathcal{E}(S^*) \cap \Delta|$ edges. Using these observations, we show that the procedure of iterating over all $x \in C$ and computing the densest subgraph in G_x will lead to an $O(\log(n_U) \log^d(n_V))$ -approximation algorithm for the WBC problem in polynomial time.

For the GWBC problem, we proceed in a similar manner. As before, we reduce GWBC to an instance of the weighted set cover problem; however, in this case, the weight of each biclique (set) is defined as the number of its vertices plus the parameter c . Due to the presence of the parameter c , the classical densest subgraph formulation is no longer appropriate. Instead, we introduce the *c-densest subgraph* problem, in which, given a graph, the objective is to find a subgraph that minimizes the ratio of the number of vertices plus c to the number of edges. We show how known results in graph theory yield efficient exact or approximate solutions for the *c-densest subgraph* problem.

3.1 Expensive approx. algorithm for GWBC

Given a bipartite graph $G(V, U, E)$, we map GWBC to the well-known *weighted set cover problem*. In the weighted set cover problem we are given a universe $\mathcal{A}$ of N items and a family of subsets $T = \{T_1, \dots, T_M\}$ of $\mathcal{A}$ such that each subset T_i is associated with a weight $w_i \geq 0$. The goal is to select a family of sets $T' \subseteq T$ such that every element $a \in \mathcal{A}$ is covered by at least one set in T' and $\sum_{T_i \in T'} w_i$ is minimized. It is known that a greedy algorithm that in each iteration selects the set T_i with the smallest ratio $\frac{w_i}{b_i}$, where b_i is the number of uncovered items in $\mathcal{A}$ contained in T_i , is an $O(\log N)$ -approximation for the weighted set cover problem [20].

We can straightforwardly map our problem to an instance of the weighted set cover problem. The universe is defined as $\mathcal{A} = E$, i.e., all edges we must cover. For every biclique S in G we construct a set $T_S = \mathcal{E}(S)$ with weight $w_S = c + \mathcal{D}(S)$. For every set cover in the set system $(\mathcal{A}, T)$ of weight W there exists a biclique cover S in G with cost $\sigma(S) = W$, and vice versa. Thus, the greedy algorithm that chooses in each iteration the biclique S that minimizes $\frac{w_S}{b_S}$, where b_S is the number of uncovered edges in $\mathcal{E}(S)$, returns an $O(\log n_U)$ -approximation for the GWBC problem. Even more generally, any algorithm that chooses in each iteration a biclique S such that $\frac{w_S}{b_S} \leq \hat{\gamma} \cdot \min_{S'} \frac{w_{S'}}{b_{S'}}$, returns an $O(\hat{\gamma} \cdot \log n_U)$ -approximation for the GWBC problem. In the next section, for δ -disk bipartite graphs we show how we can compute in polynomial time an $O(\log^d n_V)$ -approximation of the biclique with the minimum ratio of weight over number of newly covered elements.

3.2 Fast approx. algorithm for GWBC

Before describing our algorithm, we first introduce two technical tools that are central to our approach.

3.2.1 Range tree [24]. A range tree is tree-based geometric data structure that is used for range aggregation queries in $\mathbb{R}^d$, where $d = O(1)$. While a range tree has various interesting properties, we only mention the main property that we need for our algorithm. Let P be a set of points in $\mathbb{R}^d$ and let $\mathcal{T}$ be the range tree constructed over P . The range tree $\mathcal{T}$ constructs a set C of $O(|P| \cdot \log^{d-1}(|P|))$ canonical axis-aligned hyper-rectangles in $\mathbb{R}^d$. Let ρ be any hyper-rectangle in $\mathbb{R}^d$. We know that there exists a subset of $O(\log^d |P|)$ non-intersected hyper-rectangles $C(\rho) \subseteq C$ such that $\bigcup_{x \in C(\rho)} (P \cap x) = P \cap \rho$. For every $x \in C$ let $\bar{x}$ be the minimum volume axis-aligned hyper-rectangle such that $x \cap P = \bar{x} \cap P$. Without loss of generality, we assume every x in C is minimal, i.e., $x = \bar{x}$.

3.2.2 c-densest subgraph. Given a bipartite graph $G(V, U, E)$, the problem of computing the densest subgraph asks to compute a subgraph $X \subseteq G$ such that $\frac{|\mathcal{E}(X)|}{|\mathcal{D}(X)|}$ is maximized. Two common techniques to solve the densest subgraph problem are the max-flow based algorithm [31], and an LP-based algorithm [15]. There exists also a greedy 2-approximation algorithm that runs in time linear to the size of the input graph [15]. For the GWBC problem we are interested in a variation of the densest subgraph problem that we call *c-densest subgraph problem*. Given a bipartite graph $G(V, U, E)$ and a real number $c \geq 0$, the goal is compute a subgraph $X \subseteq G$ such that $\frac{|\mathcal{E}(X)|}{|\mathcal{D}(X)| + c}$ is maximized.

Algorithm 1: RT-BC

Input: A δ -disk bipartite graph $G(V, U, E)$ in ℓ_∞^d , and $c \geq 0$
Output: A biclique edge cover $\mathcal{S}$ of G

- 1 $\mathcal{S} \leftarrow \emptyset, \quad i \leftarrow 1;$
- 2 Initialize hash table B with $B(v, u) \leftarrow 1$ for every $(v, u) \in E$;
- 3 Build $\mathcal{T}$ on V and let C be its canonical hyper-rectangles;
- 4 **while** there exists $(v, u) \in E$ with $B(v, u) = 1$ **do**
- 5 $\phi^* \leftarrow +\infty, \quad S' \leftarrow \emptyset;$
- 6 **foreach** $x \in C$ **do**
- 7 $V_x \leftarrow V \cap x, \quad U_x \leftarrow \{u \in U \mid x \subseteq \rho_u\};$
- 8 $E_x \leftarrow \{(v, u) \in E \mid v \in V_x, u \in U_x, B(v, u) = 1\};$
- 9 Construct $G_x(V_x, U_x, E_x);$
- 10 $S^{(x)} \leftarrow \text{DenSub}(G_x, c), \quad b_x \leftarrow |E(S^{(x)})|;$
- 11 **if** $b_x = 0$ **then continue;**
- 12 $\hat{\phi} \leftarrow \frac{|\mathcal{D}(S^{(x)})| + c}{b_x};$
- 13 **if** $\hat{\phi} < \phi^*$ **then** $\phi^* \leftarrow \hat{\phi}, \quad S' \leftarrow S^{(x)};$
- 14 Define biclique S_i induced by $\mathcal{D}(S')$;
- 15 **foreach** $v \in \mathcal{V}(S_i), u \in \mathcal{U}(S_i)$ **do**
- 16 $B(v, u) \leftarrow 0;$
- 17 $\mathcal{S} \leftarrow \mathcal{S} \cup \{S_i\};$
- 18 $i \leftarrow i + 1;$
- 19 **return** $\mathcal{S};$

Due to space constraints, we defer to Appendix B.1 the details of how the known exact max-flow-based algorithm for the standard densest subgraph problem can be extended to obtain an exact polynomial-time algorithm for the c -densest subgraph problem. For our purposes even a $O(1)$ -approximation algorithm for the c -densest subgraph problem is sufficient. Putting everything together, we get the next result.

Lemma 3.1. *Given a bipartite graph $G(V, U, E)$ and a real number $c \geq 0$, there exists an exact algorithm for the c -densest subgraph problem that runs in $O((|V| + |U|) \cdot |E| \cdot \log(\max\{|V|, |U|\}))$ time. There also exists an $O(1)$ -approximation algorithm with running time $O(|V| + |E|)$.*

Let $\text{DenSub}(G, c)$ denote the procedure that returns an $O(1)$ -approximation of the c -densest subgraph problem on a bipartite graph G .

3.2.3 Approximation algorithm. As we mentioned in Subsection 3.1 we will implement an approximate version of the greedy algorithm, where in each iteration i it computes a biclique S_i such that $\frac{w_{S_i}}{b_{S_i}} \leq O(\log^d n) \min_{S'_i} \frac{w_{S'_i}}{b_{S'_i}}$, where $w_{S_i} = |\mathcal{D}(S_i)| + c$ and b_{S_i} is the number of edges in $\mathcal{E}(S_i)$ that have not been covered by bicliques already selected in the previous iterations.

We initialize a hash table B such that for each edge $(v, u) \in E$, we set $B(v, u) = 1$ if (v, u) is not yet covered by the current biclique cover $\mathcal{S}$, and $B(v, u) = 0$ otherwise. We also construct a range tree $\mathcal{T}$ over V and let C be the set of its canonical hyper-rectangles.

Next, consider any iteration of the greedy algorithm, where $\mathcal{S}$ contains the currently selected bicliques and hash-table B is up to dated. For example, consider the beginning of the i -th iteration

where $\mathcal{S} = \{S_1, \dots, S_{i-1}\}$. We go through every canonical hyper-rectangle $x \in C$. We construct the bipartite graph $G_x(V_x, U_x, E_x)$ where $V_x = V \cap x$, $U_x = \{u \in U \mid x \subseteq \rho_u\}$, and $E_x = \{(v, u) \in E \mid v \in V_x, u \in U_x, B(v, u) = 1\}$. Recall that ρ_u is a box in $\mathbb{R}^d$ with center u and equal side lengths 2δ . In other words, we create the bipartite graph that contains all vertices from V in x , all vertices from U whose boxes completely cover x and all uncovered edges between these vertices. We call $\text{DenSub}(G_x, c)$ and let $S^{(x)}$ be the returned subgraph of G_x . We set $x_i = \arg \min_{x \in C} \frac{|\mathcal{D}(S^{(x)})| + c}{b_{S^{(x)}}}$, and let S_i be the biclique such that $\mathcal{D}(S_i) = \mathcal{D}(S^{(x_i)})$ and $\mathcal{E}(S_i)$ consists of all edges in E between $\mathcal{V}(S^{(x_i)})$ and $\mathcal{U}(S^{(x_i)})$. As we show in the correctness proof, S_i is a valid biclique of G . For every $v \in \mathcal{V}(S_i)$ and $u \in \mathcal{U}(S_i)$ we set $B(v, u) = 0$. We add S_i in $\mathcal{S}$ and continue with the next iteration of the greedy algorithm. The greedy algorithm terminates when all edges are covered, i.e., $B(v', u') = 0$ for every edge $(v', u') \in E$. We present the pseudocode of our algorithm in Algorithm 1.

Correctness and analysis. Recall that without loss of generality we consider that $n_V \leq n_U$.

Lemma 3.2. $\sigma(\mathcal{S}) \leq O(\log(n_U) \log^d(n_V)) \sigma(\mathcal{S}^*)$, where $\mathcal{S}^*$ is the biclique edge cover that minimizes $\sigma(\mathcal{S}^*)$.

PROOF. From Subsection 3.1 it is sufficient to show that in each iteration i of our algorithm we select a biclique S_i such that $\frac{|\mathcal{D}(S_i)| + c}{b_{S_i}} \leq O(\log^d n_V) \frac{|\mathcal{D}(S_i^*)| + c}{b_{S_i^*}}$, where $b_{S_i} = |\mathcal{E}(S_i) \setminus (\bigcup_{j \in [i-1]} \mathcal{E}(S_j))|$ and S_i^* is the biclique that minimizes $\frac{|\mathcal{D}(S_i^*)| + c}{b_{S_i^*}}$.

First, it is straightforward to see that S_i is a valid biclique in G . In fact, for every $x \in C$ if $v \in \mathcal{V}(S^{(x)})$ and $u \in \mathcal{U}(S^{(x)})$, it holds that $(v, u) \in E$. Indeed, notice that by the construction of V_x and U_x , for every $v \in V_x$ and $u \in U_x$, $\psi(v, u) \leq \delta$. Hence for every vertex u in the subset of V_x that is selected in $S^{(x)}$ and every vertex u in the subset of U_x that is selected in $S^{(x)}$ it holds that $(v, u) \in E$.

We note that $y = \bigcap_{u \in \mathcal{U}(S_i^*)} \rho_u$ is a hyper-rectangle in $\mathbb{R}^d$ and since S_i^* is a biclique it holds that $\mathcal{V}(S_i^*) \subseteq y$. From the definition of a range tree, let $C(y)$ be the set of $O(\log^d n_V)$ canonical hyper-rectangles of $\mathcal{T}$ that cover all points in $\mathcal{V}(S_i^*)$. For every hyper-rectangle $z \in C(y)$ let $\xi_z = |(\mathcal{V}(S_i^*) \cap z) \times \mathcal{U}(S_i^*)| \setminus (\bigcup_{j \in [i-1]} \mathcal{E}(S_j))|$, i.e., the number of uncovered edges among vertices of $\mathcal{V}(S_i^*)$ in z and vertices of $\mathcal{U}(S_i^*)$. By definition notice that $b_{S_i^*} = \sum_{z \in C(y)} \xi_z$. Let z' be the canonical hyper-rectangle in $C(y)$ such that $\xi_{z'} = \max_{z \in C(y)} \xi_z$. Since there are $O(\log^d n_V)$ canonical hyper-rectangles in $C(y)$, we have that $\xi_{z'} \geq \frac{b_{S_i^*}}{O(\log^d n_V)}$. Furthermore, $|\mathcal{V}(S_i^*) \cap z'| \leq |\mathcal{V}(S_i^*)|$. Hence,

$$\frac{|\mathcal{V}(S_i^*) \cap z'| + |\mathcal{U}(S_i^*)| + c}{\xi_{z'}} \leq O(\log^d n) \frac{|\mathcal{D}(S_i^*)| + c}{b_{S_i^*}}. \quad (1)$$

Next, notice that z' is a canonical hyper-rectangle in C that our algorithm visits and computes the subgraph $S^{(z')}$. By definition,

$$\frac{|\mathcal{D}(S_i)| + c}{b_{S_i}} \leq \frac{|\mathcal{D}(S^{(z')})| + c}{b_{S^{(z')}}}. \quad (2)$$

Finally, notice that $V_{z'} \supseteq \mathcal{V}(S_i^*) \cap z', U_{z'} \supseteq \mathcal{U}(S_i^*)$ and by construction, $E_{z'}$ contains all uncovered edges between the vertices in

$V_{z'}$ and $U_{z'}$. The procedure $\text{DenSub}(G_{z'}, c)$ computes the subgraph $S^{(z')} \subseteq G_{z'}$ that approximately maximizes the ratio $\frac{|\mathcal{E}_{G_{z'}}(S^{(z')})|}{|\mathcal{D}(S^{(z')})| + c}$, up to a constant factor, where $\mathcal{E}_{G_{z'}}(S^{(z')})$ are the edges in $E_{z'}$ between nodes in $\mathcal{V}(S^{(z')})$ and $\mathcal{U}(S^{(z')})$. By definition every edge in $E_{z'}$ is an uncovered edge of E , so equivalently $\text{DenSub}(G_{z'}, c)$ computes the subgraph $S^{(z')} \subseteq G_{z'}$ that approximately minimizes $\frac{|\mathcal{D}(S^{(z')})| + c}{|\mathcal{E}_{G_{z'}}(S^{(z')})|} = \frac{|\mathcal{D}(S^{(z')})| + c}{b_{S^{(z')}}}$, up to a constant multiplicative factor. Hence, by the near-optimality of $\text{DenSub}(G_{z'}, c)$ we have that

$$\frac{|\mathcal{D}(S^{(z')})| + c}{b_{S^{(z')}}} \leq O(1) \cdot \frac{|\mathcal{V}(S_i^*) \cap z'| + |\mathcal{U}(S_i^*)| + c}{\xi_{z'}}. \quad (3)$$

From Equations (1), (2), and (3), we get

$$\frac{|\mathcal{D}(S_i)| + c}{b_{S_i}} \leq O(\log^d n_V) \cdot \frac{|\mathcal{D}(S_i^*)| + c}{b_{S_i^*}}.$$

Lemma 3.3. *Our algorithm runs in $O(n_V \cdot (n_U + m) \cdot m \cdot \log^{d-1}(n_V))$ time and uses $O(n_U + m + n_V \cdot \log^{d-1} n_V)$ space.*

PROOF. The range tree $\mathcal{T}$ is constructed in $O(n_V \log^{d-1} n_V)$ and there are $O(n_V \log^{d-1} n_V)$ canonical hyper-rectangles. For each $x \in C$ we create the graph G_x in $O(n_V + n_U + m)$ time and the procedure $\text{DenSub}(G_x, c)$ is executed in $O(n_U + m)$ time (Lemma 3.1). Hence each iteration of our greedy algorithm runs in $O(n_V \cdot (n_U + m) \cdot \log^{d-1}(n_V))$ time. In each iteration of the greedy algorithm at least one new edge is covered so the overall time of our algorithm is bounded by $O(n_V \cdot (n_U + m) \cdot m \cdot \log^{d-1}(n_V))$. Finally, our algorithm constructs the range tree $\mathcal{T}$ with space $O(n_V \log^{d-1} n_V)$. All the other steps of the algorithm including the algorithm for the c -densest subgraph problem (Lemma 3.1) use linear space with respect to the size of the graph G . $\square$

Putting everything together we conclude to Theorem 3.4.

THEOREM 3.4. *Given a δ -disk bipartite graph $G(V, U, E)$ in ℓ_∞^d , where $d = O(1)$, such that $n_V = |V|$, $n_U = |U|$ (and without loss of generality $n_V \leq n_U$), and $|E| = m$, there exists an $O(\log(n_U) \cdot \log^d(n_V))$ -approximation algorithm for the GWBC (and WBC) problem for any $c \geq 0$, that runs in $O(n_V \cdot (n_U + m) \cdot m \cdot \log^{d-1}(n_V))$ time and uses $O(n_U + m + n_V \log^{d-1} n_V)$ space.*

4 Extensions

We show how the algorithm from Subsection 3.2.3 can be extended to intersection bipartite graphs. We also describe how to adapt the algorithm to δ -disk bipartite graphs in ℓ_α^d for $\alpha \geq 1$.

4.1 Intersection bipartite graphs

Let $G(V, U, E, \mathcal{R}_V, \mathcal{R}_U)$ be an intersection bipartite graph. Our algorithm is an extension of the algorithm in Section 3.2.3. The high level idea is that for each hyper-rectangle $r_v \in \mathcal{R}_V$ we construct a point p_v in $\mathbb{R}^{2d}$ and for each r_u a hyper-rectangle $\hat{p}_u$ in $\mathbb{R}^{2d}$, such that $r_v \cap r_u \neq \emptyset$ if and only if $p_v \in \hat{p}_u$. A hyper-rectangle r in $\mathbb{R}^d$ is defined as $r = [r^{(1)-}, r^{(1)+}] \times \dots \times [r^{(d)-}, r^{(d)+}]$, where $[r^{(j)-}, r^{(j)+}]$ is the interval of r in j -th dimension. For every $v \in V$ let p_v be the point in $\mathbb{R}^{2d}$ defined by the corners of r_v : $p_v = (r_v^{(1)-}, \dots, r_v^{(d)-}, r_v^{(1)+}, \dots, r_v^{(d)+})$. Let $P = \{p_v \mid v \in V\}$ and let $\mathcal{T}$ be the range tree constructed on P . For every $r_u \in \mathcal{R}_V$, we

construct a hyper-rectangle $\hat{p}_u = (-\infty, r_u^{(1)+}] \times (-\infty, r_u^{(2)+}] \times \dots \times (-\infty, r_u^{(d)+}] \times [r_u^{(1)-}, \infty) \times [r_u^{(2)-}, \infty) \times \dots \times [r_u^{(d)-}, \infty)$. For every $x \in C$, we define $V_x = \{v \in V \mid p_v \in x\}$, and $U_x = \{u \in U \mid x \subseteq \hat{p}_u\}$. All the other steps of the algorithm follow verbatim.

The correctness of the method and Extensions to interval and k -NN bipartite graphs are given in Appendix B.2.

THEOREM 4.1. *Given an intersection bipartite graph $G(V, U, E, \mathcal{R}_V, \mathcal{R}_U)$ in $\mathbb{R}^d$, where $d = O(1)$, such that $n_V = |V|$, $n_U = |U|$ (and without loss of generality $n_V \leq n_U$), and $|E| = m$, there exists an $O(\log(n_U) \cdot \log^{2d}(n_V))$ -approximation algorithm for the GWBC (and WBC) problem for any $c \geq 0$, that runs in $O(n_V \cdot (n_U + m) \cdot m \cdot \log^{2d-1}(n_V))$ time and uses $O(n_U + m + n_V \log^{2d-1} n_V)$ space.*

4.2 δ -disk graph for any ℓ_α^d

We next discuss how the algorithm from Theorem 3.4 extends to ℓ_α^d for any $\alpha \geq 1$. In the ℓ_∞^d metric, an edge $(v, u) \in E$ exists if and only if v lies in an axis-aligned box ρ_u of side length 2δ centered at u . More generally, for any α , we define the ball $\bigcirc_u = \{p \in \mathbb{R}^d \mid \psi_\alpha(u, p) \leq \delta\}$, and observe that $(v, u) \in E$ if and only if $v \in \bigcirc_u$. The correctness of our algorithm relies on range trees, which decompose any hyper-rectangle into $O(\log^d n_V)$ canonical regions. However, for $\alpha \neq \infty$, the region $\bigcirc_u$ is not a hyper-rectangle, and therefore range trees cannot be applied. We show two approaches for handling metrics ℓ_α^d for any $\alpha \geq 1$. The first is a theoretical approach based on the notion of ε -bicliques that are graph structures that approximate standard bicliques. Due to space constraints, we present this in Appendix B.3. The second is a practical approach, which we also employ in our experimental evaluation.

4.2.1 Practical approach. The theoretical approach constructs an ε -biclique edge cover rather than an exact biclique cover and relies on an advanced geometric data structure (BBD tree). In practice, however, when our goal is to compute a biclique edge cover for general metrics ℓ_α^d with $\alpha \geq 1$, we instead employ an R -tree [36]. The R -tree is one of the most widely used geometric data structures for range aggregation queries, where the query range can be any convex region. The R -tree constructed over V induces a collection C of $O(|V|)$ canonical axis-aligned hyper-rectangles in $\mathbb{R}^d$. For any convex region $\rho \subseteq \mathbb{R}^d$, there exists a subset $C(\rho) \subseteq C$ of λ pairwise disjoint hyper-rectangles such that $\bigcup_{x \in C(\rho)} (V \cap x) = V \cap \rho$. While for range trees and BBD-trees the parameter λ is provably bounded, in the case of R -trees it can be as large as $O(n_V)$ in the worst case. Nevertheless, in practice λ is typically much smaller and is often observed to be $O(\log n_V)$. We refer to λ as the *range cover size*.

For any given $\alpha \geq 1$, we construct an R -tree over V and execute the algorithm from Subsection 3.2.3, iterating over the canonical hyper-rectangles of the R -tree instead of those of the range tree. All other steps of the algorithm remain unchanged. Using the same analysis, but assuming a range cover size of λ instead of $O(\log^d n_V)$, we obtain the following result.

THEOREM 4.2. *Given a δ -disk bipartite graph $G(V, U, E)$ in ℓ_α^d , where $\alpha \geq 1$, such that $n_V = |V|$, $n_U = |U|$, and $|E| = m$, there exists an $O(\lambda \cdot \log(n_U))$ -approximation algorithm for the GWBC (and WBC) problem for any $c \geq 0$, where λ is the range cover size, that runs in $O(n_V \cdot (n_U + m) \cdot m)$ time and uses $O(n_U + n_V + m)$ space.*

Remark 1. Due to its practicality in all our experiments we use the algorithm from Theorem 4.2.

Remark 2. The running times stated in our theorems are loose worst-case upper bounds. In practice, performance is significantly better. Specifically, each greedy iteration typically covers many edges (rather than a single edge), the edges in G_x are much smaller than m , and only a small subset of canonical hyper-rectangles needs to be examined per iteration. If the greedy algorithm runs for κ_1 iterations, each G_x contains at most κ_2 edges, and at most κ_3 hyper-rectangles are visited per iteration, a more realistic running time is $O(\kappa_3(n_U + \kappa_2)\kappa_1)$.

5 Experiments

In this section, we implement the practical algorithm from Theorem 4.2 and compare its effectiveness and efficiency against standard baseline methods for WBC and GWBC on both real and semi-synthetic datasets. Specifically, for each algorithm we evaluate the following metrics: (i) the total size of the returned biclique edge cover, (ii) the maximum memory consumption, and (iii) the running time required to compute a biclique edge cover. All datasets and our code can be found in [5].

5.1 Datasets

We evaluate our algorithms on three real geometric datasets **Adults** [10], **Default of Credit Card Clients** [63], **MAGIC Gamma Telescope** [11], two real bipartite graphs **MovieLens100K** [33], **MovieLens1M** [34], two matrix datasets **WorldCities**, **162bit** from [22] corresponding to real bipartite graphs, and one semi-synthetic dataset **POPSIM** [49].

In the datasets **Adults**, **Credit**, **Gamma**, and **POPSIM**, each data point lies in $\mathbb{R}^d$ and is associated with a categorical attribute. In all cases, we normalize the coordinates so that they lie in the range $[0, 1]$. We select a categorical attribute to partition the input points into two disjoint sets, V and U , thereby defining the bipartition. We consider five values of δ and construct the corresponding δ -disk bipartite graphs. For a fixed value of δ , a vertex $v \in V$ and a vertex $u \in U$ are connected by an edge if $\psi_\alpha(v, u) \leq \delta$. In our experiments, we use the ℓ_2^d and ℓ_∞^d metric space ($\alpha = \{2, \infty\}$). For **POPSIM**, we construct two additional geometric bipartite graphs: a large and very sparse graph, called **POPSIM-S**, in which the average degree is less than 2, and a large and very dense graph, called **POPSIM-D**, in which the average degree is greater than 160. We show all statistics of our datasets in Table 1.

While **Adults**, **Credit**, **Gamma**, and **POPSIM** datasets are inherently geometric, this is not the case for the real bipartite graphs **MovieLens100K** and **MovieLens1M**. In these datasets, one side of the bipartite graph corresponds to users, the other to movies, and edges represent user ratings. Similarly, the bipartite graphs derived by the matrix datasets **WorldCities** and **162bit** are not geometric. However, our algorithm RT-BC operates on geometric bipartite graphs. Therefore, we embed the original bipartite graphs into a geometric space while attempting to preserve their edge structure with minimal distortion. More specifically, given a bipartite graph $G(U, V, E)$, we construct an embedding G' in $\mathbb{R}^d$, for a sufficiently large dimension d , such that every $u \in U$ and $v \in V$ is mapped to points $p_u, p_v \in \mathbb{R}^d$, and edges are induced by a distance threshold, i.e., $(u, v) \in E \iff \psi_\alpha(p_u, p_v) \leq r$. Our objective is to ensure that G' approximates G as closely as possible, so that the experimental

Dataset	δ	m	n	d	Group Attribute
Adults	0.05	47,963	14,846	5	Sex
	0.1	445,724			
	0.15	1,415,555			
	0.2	3,016,343			
Credit	0.25	5,498,540	28,726	6	Sex
	0.002	889,671			
	0.003	1,739,709			
	0.004	2,710,136			
	0.005	3,750,745			
Gamma	0.006	4,824,359	18,905	10	Class
	0.13	194,737			
	0.14	284,626			
	0.15	399,339			
	0.16	541,050			
POPSIM	0.17	711,158	8,591	2	Race
	0.04	1,468,984			
	0.06	2,660,732			
	0.08	3,801,451			
	0.1	4,857,704			
POPSIM-S	-	5,625,395	3,211,965	2	Race
POPSIM-D	-	122,880,782	755,182	2	Race
MovieLens100K	-	100,000	2,700	256	-
MovieLens1M	-	1,000,000	10,000	256	-
WorldCities	-	7,518	415	256	-
162bit	-	37,118	7,203	256	-

Table 1: Statistics of δ -disk graphs in our experiments for ℓ_2^d .

evaluation reflects the performance of RT-BC rather than artifacts introduced by the embedding process. In practice, we fix $\alpha = 2$ (Euclidean distance) and $r = 1$, and optimize over the embedding coordinates in dimension d using a gradient-based optimization procedure. We enforce that all original edges of G are preserved in G' , while minimizing the number of additional induced edges, ideally introducing none. Our approach is inspired by the geometric graph embedding framework of [45].

For both **MovieLens100K** and **MovieLens1M**, we construct embeddings in $\mathbb{R}^{256}$ that are highly accurate, introducing only at most 18 additional edges overall. For **WorldCities** and **162bit**, we construct embeddings in $\mathbb{R}^{256}$ introducing no additional edges.

We report summary statistics (including the number of vertices, number of edges, dimensionality, and grouping attribute) for all datasets (for $\alpha = 2$) and all values of δ in Table 1. We also show the statistics for $\alpha = \infty$ in Table 3.

5.2 Algorithms

We describe the algorithms compared in our experiments.

Our algorithm: RT-BC. We implemented our practical algorithm (with provable guarantees) from Theorem 4.2. To generate the canonical hyper-rectangles we use Scipy’s KDTree implementation of R-trees [36]. For the WBC problem, we use Charikar’s 2-approximation algorithm [15] to compute the densest subgraph, as implemented in [3]. For the GWBC problem, we modified the implementation of the exact algorithm [31] for the densest subgraph problem, as shown in Section 3 to solve the c -densest subgraph.

Baselines. For each WBC and GWBC problem, we compare our approach against three baselines. The first two baselines share the same high level approach: They are greedy algorithms and run in two phases. In the first phase they generate a candidate set of bicliques. In the second phase they run a greedy algorithm to

compute a biclique edge cover using the selected candidate bicliques. The third baseline is the CPGR algorithm from [16].

- **Role**: This is the state-of-the-art heuristic for the edge-RMP problem in role-mining [46, 58], which is equivalent to WBC for bipartite graphs. It uses the RoleMiner algorithm [59] to generate a candidate set of bicliques.

- **AMBEA**: It constructs a candidate set of bicliques by enumerating all maximal bicliques using AMBEA [53].

- **CPGR**: This is the state-of-the-art algorithm for constructing a biclique partition, as implemented in [16].

Details of Role and AMBEA. Following candidate generation, the first two baselines employ the same greedy selection criteria. For the WBC problem, the greedy procedure chooses in each iteration the biclique that minimizes the ratio of the number of nodes over the number of uncovered edges. For the GWBC problem, the greedy procedure chooses the biclique that minimizes the ratio of the number of nodes plus c over the number of uncovered edges.

We initially observed that, in all cases, Role and AMBEA run significantly slower than our algorithm and frequently exceed available memory limits. This behavior is primarily due to the extremely large number of candidate bicliques generated by the baselines. This behavior is consistent with prior observations. For example, in [58], the authors generated synthetic datasets with fewer than 400 vertices and reported more than 6,000 candidate bicliques, with running times exceeding 60 seconds. In our setting, the effect is even more pronounced: On the **Adults** dataset in ℓ_∞^d with $\delta = 0.1$, Role requires 5,473 seconds, generates 1,130,246 candidate bicliques, and reaches a peak memory usage of 42,068 MB.

To ensure a fair and feasible comparison, we introduce an early-stopping criterion for Role and AMBEA. At a high level, we constrain all algorithms to operate within comparable time budgets. Specifically, we allow baselines to spend between 10% and 35% of our algorithm’s total execution time generating candidate bicliques. Under this setting, both baselines still require slightly more total running time than our algorithm. Consequently, our evaluation grants the baselines an advantage by allowing them to run longer and generate more candidate bicliques, while remaining within feasible time and memory limits. Additionally, the consecutively generated candidates by both of our baselines tend to share many vertices. Thus, to ensure the quality of candidates, we also randomly shuffle the vertex order before generating them.

Details of CPGR. We observed that across all datasets we evaluated, CPGR produced representations whose sizes were close to the size of the original graph. In other words, the algorithm was unable to identify many non-trivial biclique partitions beyond simple edge-based decompositions that lead to compact representations. This behavior is expected, since biclique partitioning is substantially more restrictive than biclique covering (every biclique partition is a biclique cover, but not vice versa), and the objective in [16] is not to compute a minimum-size bipartite partition. Consequently, we omitted this baseline from most of our figures, although we still report some representative examples of the sizes of its computed representations.

Setup. All experiments were conducted on a Windows 11 workstation equipped with an Intel Core i7-9700 CPU and 32 GB of DDR4 RAM. All algorithms were implemented in Python 3.12.9.

The AMBEA algorithm [53] used in AMBEA, however, is a parallel implementation written in C++. It was compiled as a standalone executable and invoked via Python subprocesses. As a result, AMBEA enjoys an additional performance advantage compared to both Role and RT-BC. Memory usage was monitored using the Python library `psrecord`, with a 3-second polling interval to minimize measurement overhead.

5.3 Experimental results for WBC

In this section, we first provide a comprehensive comparison of our algorithm (RT-BC) against Role and AMBEA on all versions of **Adults**, **Credit**, **Gamma**, and **POPSIM**, for different values of δ , as reported in Table 1. For each setting, we report the size of the returned representation, the peak memory usage during execution, and the running time of each algorithm. We omit CPGR from these experiments because, in all cases, it returns representations that are substantially larger than those of the other baselines, with sizes close to that of the original graph. We discuss the performance of CPGR separately later.

Experiments are conducted under both ℓ_2 and ℓ_∞ metrics. Since the observed trends are consistent across the two settings, we present the results for the ℓ_2 metric here and defer the ℓ_∞ results to Appendix C. As discussed above, to ensure a fair comparison, we apply an early-stopping criterion that allows the baselines to run slightly longer than our algorithm, as illustrated in Figure 3(c). This enables a direct comparison of the quality of the resulting representations, as well as the peak memory consumption, under comparable time budgets.

As shown in Figure 3(a), our algorithm RT-BC consistently outperforms both Role and AMBEA in terms of representation size across all datasets and for all values of δ . The only exception occurs for the **Adult** dataset with $\delta = 0.05$, where all algorithms produce representations of comparable size. In this instance, the corresponding bipartite graph is sparse, with an average degree of approximately 3.4 per vertex, leaving limited room for improvement. As the graphs become larger and denser, however, the advantage of RT-BC becomes increasingly pronounced. For example, on real-world datasets, RT-BC produces representations that are up to four times smaller than those returned by both baselines (e.g., on the **Credit** dataset). The improvement is even more substantial on the semi-synthetic **POPSIM** dataset, where the representation produced by RT-BC is approximately 26 times smaller than those produced by the baselines. This behavior can be attributed to the low intrinsic dimensionality of the **POPSIM** dataset and the highly geometric manner in which it is generated. The R -tree data structure employed by RT-BC performs particularly well in low-dimensional settings. By exploring the canonical hyper-rectangles generated by the R -tree, the algorithm is able to identify regions x whose corresponding subgraphs G_x capture near-optimal bicliques, leading to significantly more compact representations.

Next, in Figure 3(b), we compare the peak memory usage across all datasets. RT-BC consistently uses significantly less memory than both baselines. In some cases, our method requires up to three times less memory than the baselines (e.g., **Gamma** dataset). These results are consistent with our theoretical analysis. As shown in Theorem 4.2, our algorithm uses asymptotically only linear space with respect to the size of the input graph. In contrast, the memory

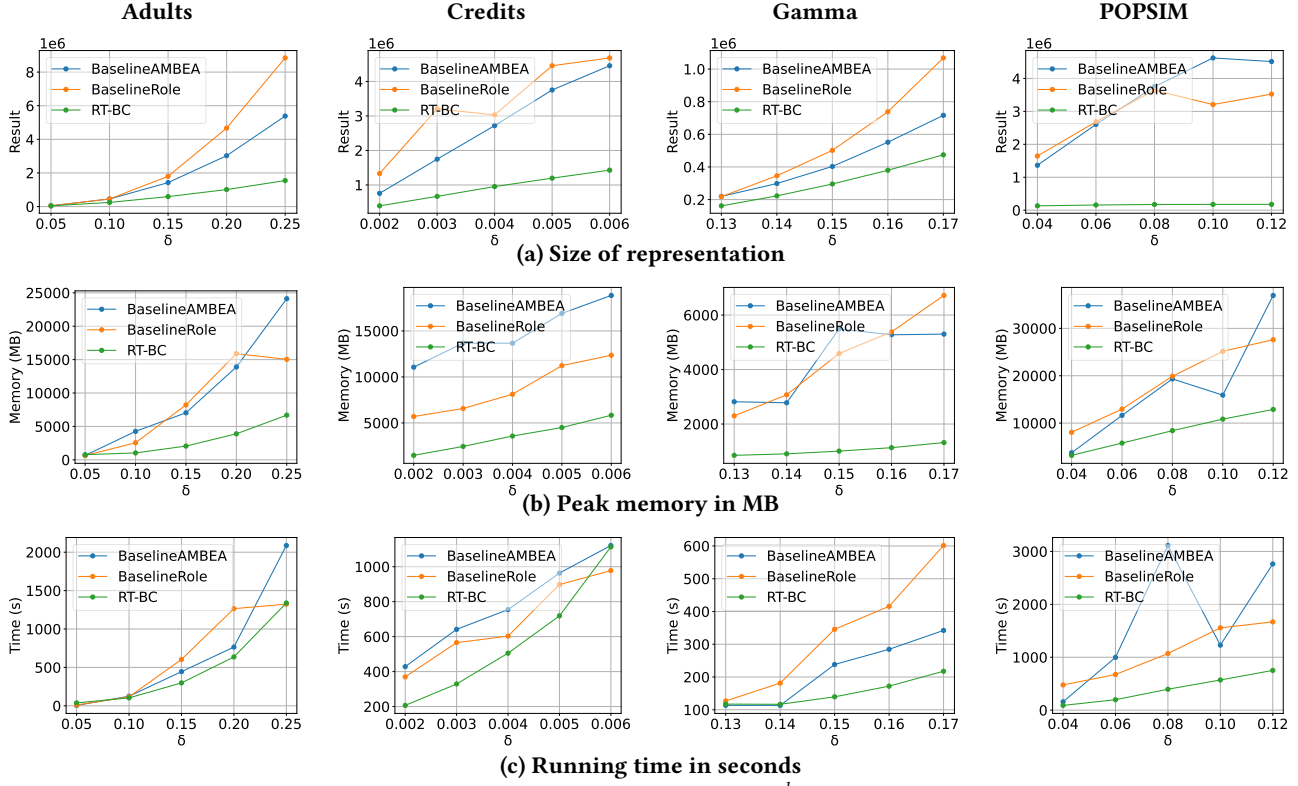


Figure 3: Experimental results for WBC in ℓ_2^d across all datasets

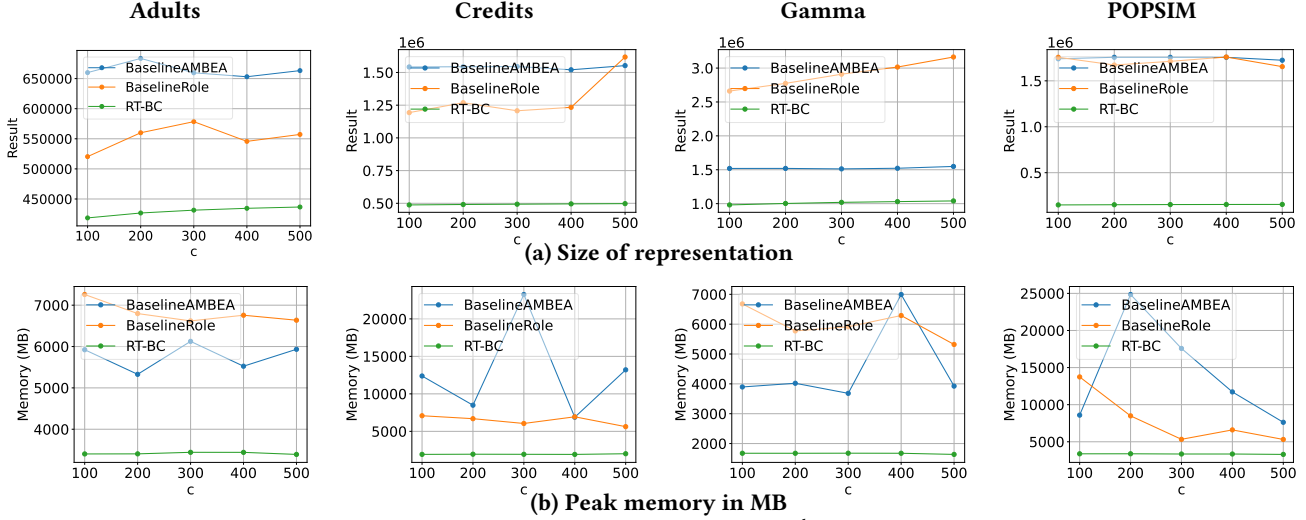


Figure 4: Experimental results for GWBC in ℓ_∞^d across all datasets

Baseline/Dataset	Adults $\delta = 0.2$	Gamma $\delta = 0.16$	POPSIM-S	POPSIM-D	MovieLens100K	MovieLens1M	WorldCities	162bit
Role	4,668,513	738,232	7,038,915	108,138,671	188,227	2,336,652	13,429	52,577
AMBEA	3,024,324	551,686	5,387,872	122,279,423	108,014	1,079,149	7,427	51,368
CPGR	5,242,099	1,082,100	11,250,790	245,761,564	200,000	2,000,000	15,034	74,236
Original	6,032,686	1,082,100	11,250,790	245,761,564	200,000	2,000,000	15,034	74,236
RT-BC	1,012,402	379,675	3,396,491	904,425	101,152	952,869	6,034	40,590

Table 2: Representation size produced by each baseline for WBC on each dataset.

consumption of the baselines is dominated by the total size of the candidate bicliques they generate, which is typically very large.

Finally, Role and AMBEA exhibit similar behavior, producing biclique edge covers of comparable size. In some cases, AMBEA outperforms Role due to its parallel C++ implementation of the candidate biclique generation step, which allows it to explore more candidates within the same time budget.

Comprehensive comparison of all baselines on small, large, sparse, and dense bipartite graphs. Table 2 reports the representation size returned by RT-BC and all baselines (Role, AMBEA, and CPGR) across a diverse collection of bipartite graphs; for reference, we also report the size of the original edge-based representation, obtained without applying any compression algorithm. To ensure a fair comparison, we configure all baselines so that their running times are roughly comparable to that of RT-BC. The datasets in this comparison consist of two representative geometric bipartite graphs, **Adults** and **Gamma**; two variants of **POPSIM**, capturing both sparse and very large dense instances; and four real, non-geometric bipartite graphs, namely **MovieLens100K**, **MovieLens1M**, **WorldCities**, and **162bit**, after first embedding them into a geometric space. Across all real bipartite graphs, RT-BC consistently produces smaller representations than all compared baselines. Furthermore, as we mentioned earlier in this section CPGR often returns a representation whose size is essentially equal to that of the original graph, indicating that biclique partitioning is not an effective approach for WBC in these instances. This behavior is expected, since constructing a biclique partition is significantly more restrictive than constructing a biclique cover.

5.4 Experimental results for GWBC

In Figure 4, we report the representation size and peak memory usage of our algorithm and the baselines for the GWBC problem under the ℓ_∞ metric, for varying values of c and a fixed value of δ . Specifically, we use $\delta = 0.1$ for **Adult**, $\delta = 0.002$ for **Credit**, $\delta = 0.13$ for **Gamma**, and $\delta = 0.04$ for **POPSIM**. Fixing δ isolates the effect of c on the quality and memory footprint of the resulting representations. As in the WBC experiments, all algorithms are evaluated under comparable time budgets to ensure a fair comparison. Since the running-time trends are consistent with those observed for WBC, we defer the detailed timing results to Appendix C (Figure 6).

Consistent with our earlier observations, our algorithm RT-BC outperforms both baselines across all evaluation metrics, consistently producing smaller representations while using less memory. Notably, the performance gap between our method and the baselines becomes even more pronounced in the GWBC setting. For example, on the **Gamma** dataset with $\delta = 0.13$, under the WBC objective RT-BC produces a representation that is 2-times smaller than those returned by the baselines. In contrast, for the GWBC problem under the same value of δ and with $c = 500$, RT-BC returns a representation that is 3-times smaller than the baselines. Similar trends are observed for the remaining bipartite graphs, including MovieLens and the two matrix datasets; therefore, we omit them from the figures.

We observe occasional memory spikes for AMBEA. This behavior is due to the combination of early stopping and the randomized vertex ordering used at each run to avoid repeatedly exploring the same regions of the graph.

6 Related work

Lossless compact representations for graphs have been studied extensively [9, 12, 25, 26, 54]. While these techniques often achieve high compression ratios, the resulting representations are typically not directly usable by general-purpose graph algorithms and may require substantial reconstruction overhead.

Feder and Motwani [27] introduced a graph compression method that preserves path information by partitioning edges into bicliques; Chavan et al. [16] later improved its running time. In contrast, we study biclique coverings rather than partitions and explicitly formulate the optimization problem of minimizing the representation size. Unlike prior heuristic approaches, we provide approximation algorithms with provable guarantees for meaningful graph classes.

The WBC problem has been studied in theory [19, 23, 56, 57]. This body of work primarily focuses on establishing upper and lower bounds on the cost of an optimal solution. The decision version was later shown to be NP-Complete in [42, 58]. For an overview of WBC and its data mining applications, see [46]. To the best of our knowledge, no approximation algorithms or inapproximability results are known. Existing approaches are limited to heuristics without theoretical performance guarantees [46, 48, 58].

A related and well-studied variant is the bipartite dimension problem where the goal is to find the minimum number of bicliques whose union of edges covers all edges of the graph. It is known to be NP-hard [32, 51] and admits strong inapproximability results [13, 28, 35, 40, 55, 66]. On the positive side, the problem is fixed-parameter tractable [14, 29, 50] and exact polynomial-time algorithms are known for some graph classes [4, 44, 47].

A variety of problems have been studied on δ -disk graphs, including the densest subgraph problem [37], the maximum clique problem [21], and others [2, 17]. Recently, Lahn et al. [41] designed an efficient algorithm for the maximum cardinality bipartite matching problem on δ -disk bipartite graphs.

7 Future Work

Several directions remain for future work. As discussed in Section 2, for large c the GWBC problem reduces to the bipartite dimension problem, which is NP-hard to approximate [35, 55]. It is an open question whether similar inapproximability results hold for smaller values of c . Another direction is to design approximation or exact algorithms for WBC and GWBC on graph classes beyond intersection bipartite graphs. Finally, it would be interesting to integrate our representations into graph compression pipelines and evaluate the resulting compression ratios on real-world graphs.

References

- [1] Pankaj K Agarwal, Xiao Hu, Stavros Sintos, and Jun Yang. On reporting durable patterns in temporal proximity graphs. *Proceedings of the ACM on Management of Data*, 2(2):1–26, 2024.
- [2] Pankaj K Agarwal and Jiangwei Pan. Near-linear algorithms for geometric hitting sets and set covers. In *Proceedings of the thirtieth annual symposium on Computational geometry*, pages 271–279, 2014.
- [3] Heer Ambavi, Mridul Sharma, and Varun Gohil. Densest-subgraph-discovery. <https://github.com/varungohil/Densest-Subgraph-Discovery>, 2020. GitHub repository.
- [4] Jérôme Amilhastre, Marie-Catherine Vilarem, and Philippe Janssen. Complexity of minimum biclique cover and minimum biclique decomposition for bipartite domino-free graphs. *Discrete applied mathematics*, 86(2-3):125–144, 1998.
- [5] Anonymous. Datasets and code for: Compact representations of geometric bipartite graphs via weighted biclique covers. https://anonymous.4open.science/r/WBC_experiment-1547.

- [6] Sunil Arya and David M Mount. Approximate range searching. *Computational Geometry*, 17(3-4):135–152, 2000.
- [7] Sunil Arya, David M Mount, Nathan S Netanyahu, Ruth Silverman, and Angela Y Wu. An optimal algorithm for approximate nearest neighbor searching fixed dimensions. *Journal of the ACM (JACM)*, 45(6):891–923, 1998.
- [8] Aistis Atminas and Viktor Zamaraev. On forbidden induced subgraphs for unit disk graphs. *Discrete & Computational Geometry*, 60(1):57–97, 2018.
- [9] Max Bannach, Florian Andreas Marwitz, and Till Tantau. Faster graph algorithms through dag compression. In *41st International Symposium on Theoretical Aspects of Computer Science (STACS 2024)*, pages 8–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.
- [10] Barry Becker and Ronny Kohavi. Adult. UCI Machine Learning Repository, 1996. DOI: <https://doi.org/10.24432/C5XW20>.
- [11] R. Bock. MAGIC Gamma Telescope. UCI Machine Learning Repository, 2004. DOI: <https://doi.org/10.24432/C52C8B>.
- [12] Gregory Buehrer and Kumar Chellapilla. A scalable pattern mining approach to web graph compression with communities. In *Proceedings of the 2008 international conference on web search and data mining*, pages 95–106, 2008.
- [13] Parinya Chalermsook, Sandy Heydrich, Eugenia Holm, and Andreas Karrenbauer. Nearly tight approximability results for minimum biclique cover and partition. In *European Symposium on Algorithms*, pages 235–246. Springer, 2014.
- [14] Sunil Chandran, Davis Issac, and Andreas Karrenbauer. On the parameterized complexity of biclique cover and partition. In *11th International Symposium on Parameterized and Exact Computation (IPEC 2016)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2017.
- [15] Moses Charikar. Greedy approximation algorithms for finding dense components in a graph. In *International workshop on approximation algorithms for combinatorial optimization*, pages 84–95. Springer, 2000.
- [16] Akshar Chavan, Sanaz Rabinia, Daniel Grosu, and Marco Brocanelli. Speeding-up graph algorithms via clique partitioning. *arXiv preprint arXiv:2502.02477*, 2025.
- [17] Chandra Chekuri, Sarel Har-Peled, and Kent Quanrud. Fast lp-based approximations for geometric packing and covering problems. In *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1019–1038. SIAM, 2020.
- [18] Li Chen, Rasmus Kyng, Yang Liu, Richard Peng, Maximilian Probst Gutenberg, and Sushant Sachdeva. Maximum flow and minimum-cost flow in almost-linear time. *Journal of the ACM*, 72(3):1–103, 2025.
- [19] FRK Chung, P Erdős, and Joel Spencer. On the decomposition of graphs into complete bipartite subgraphs. In *Studies in Pure Mathematics: To the Memory of Paul Turán*, pages 95–101. Springer, 1983.
- [20] Vasek Chvatal. A greedy heuristic for the set-covering problem. *Mathematics of operations research*, 4(3):233–235, 1979.
- [21] Brent N Clark, Charles J Colbourn, and David S Johnson. Unit disk graphs. *Discrete mathematics*, 86(1-3):165–177, 1990.
- [22] Timothy A. Davis and Yifan Hu. The university of florida sparse matrix collection. *ACM Transactions on Mathematical Software*, 38(1):1:1–1:25, 2011. doi: 10.1145/2049662.2049663.
- [23] Akbar Davoodi, Ramin Javadi, and Behnaz Omoomi. Edge clique covering sum of graphs. *Acta Mathematica Hungarica*, 149:82–91, 2016.
- [24] Mark De Berg, Otfried Cheong, Marc Van Kreveld, and Mark Overmars. *Computational geometry: algorithms and applications*. Springer, 2008.
- [25] Laxman Dhulipala, Igor Kabiljo, Brian Karrer, Giuseppe Ottaviano, Sergey Pupyrev, and Alon Shalita. Compressing graphs and indexes with recursive graph bisection. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1535–1544, 2016.
- [26] Wenfei Fan, Yuanhao Li, Muyang Liu, and Can Lu. Making graphs compact by lossless contraction. In *Proceedings of the 2021 International Conference on Management of Data*, pages 472–484, 2021.
- [27] Tomás Feder and Rajeev Motwani. Clique partitions, graph compression and speeding-up algorithms. *Journal of Computer and System Sciences*, 51(2):261–272, 1995.
- [28] Uriel Feige and Joe Kilian. Zero knowledge and the chromatic number. *Journal of Computer and System Sciences*, 57(2):187–199, 1998.
- [29] Herbert Fleischner, Egbert Mujuni, Daniël Paulusma, and Stefan Szeider. Covering graphs with few complete bipartite subgraphs. *Theoretical Computer Science*, 410(21-23):2045–2053, 2009.
- [30] Aristides Gionis. Algorithmic methods for mining large graphs, lecture 3: Finding dense subgraphs. Lecture notes presented at Bertinoro International Spring School (BISS), 2016. Available at: <https://users.ics.aalto.fi/gionis/biss2016-lec3.pdf>.
- [31] Andrew V Goldberg. Finding a maximum density subgraph. 1984.
- [32] Lee-Ad J Gottlieb, John E Savage, and Arkady Yerukhimovich. Efficient data storage in large nanoarrays. *Theory of Computing Systems*, 38(4):503–536, 2005.
- [33] GroupLens Research. MovieLens 100K Dataset. <https://grouplens.org/datasets/movielens/100k/>, 1998. Released April 1998. Accessed May 17, 2026.
- [34] GroupLens Research. MovieLens 1M Dataset. <https://grouplens.org/datasets/movielens/1m/>, 2003. Released February 2003. Accessed May 17, 2026.
- [35] Hermann Gruber and Markus Holzer. Inapproximability of nondeterministic state and transition complexity assuming $p \neq np$. In *International Conference on Developments in Language Theory*, pages 205–216. Springer, 2007.
- [36] Antonin Guttman. R-trees: a dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD international conference on Management of data*, page 47–57, New York, NY, USA, 1984. Association for Computing Machinery. doi: 10.1145/602259.602266.
- [37] Sarel Har-Peled and Saladi Rahul. Approximating densest subgraph in geometric intersection graphs. In *42nd International Symposium on Theoretical Aspects of Computer Science (STACS 2025)*, pages 43–1, 2025.
- [38] Sarel Har-Peled and Everett Yang. Approximation algorithms for maximum matchings in geometric intersection graphs. In *38th International Symposium on Computational Geometry, SoCG 2022*, page 47, 2022.
- [39] Satyabrata Jana, Anil Maheshwari, Saeed Mehrabi, and Sasanka Roy. Maximum bipartite subgraphs of geometric intersection graphs. *International Journal of Computational Geometry & Applications*, 33(03n04):133–157, 2023.
- [40] Subhash Khot and Ashok Kumar Ponnuswami. Better inapproximability results for maxclique, chromatic number and min-3lin-deletion. In *International Colloquium on Automata, Languages, and Programming*, pages 226–237. Springer, 2006.
- [41] Nathaniel Lahn, Sharath Raghvendra, and Jiacheng Ye. A faster maximum cardinality matching algorithm with applications in machine learning. *Advances in Neural Information Processing Systems*, 34:16885–16898, 2021.
- [42] Xuemin Lin. On the computational complexity of edge concentration. *Discrete Applied Mathematics*, 101(1):197–205, 2000. URL: <https://www.sciencedirect.com/science/article/pii/S0166218X99002073>, doi: 10.1016/S0166-218X(99)00207-3.
- [43] D Lokshtanov, F Panolan, S Saurabh, J Xue, and M Zehavi. Bipartizing (pseudo-) disk graphs: Approximation with a ratio better than 3. In *International Conference on Approximation Algorithms for Combinatorial Optimization Problems (APPROX)*. LIPIcs, 2024.
- [44] Anna Lubiw. A weighted min-max relation for intervals. *Journal of Combinatorial Theory, Series B*, 53(2):151–172, 1991.
- [45] Hiroshi Maehara. Space graphs and sphericity. *Discrete Applied Mathematics*, 71(1):55–64, 1984.
- [46] Barsha Mitra, Shamik Sural, Jaideep Vaidya, and Vijayalakshmi Atluri. A survey of role mining. *ACM Comput. Surv.*, 48(4), February 2016. doi: 10.1145/2871148.
- [47] Haiko Müller. On edge perfectness and classes of bipartite graphs. *Discrete Mathematics*, 149(1-3):159–187, 1996.
- [48] Frances J Newbery. Edge concentration: A method for clustering directed graphs. In *Proceedings of the 2nd International Workshop on Software configuration management*, pages 76–85, 1989.
- [49] Khanh Duy Nguyen, Nima Shahbazi, and Abolfazl Asudeh. Popsim: An individual-level population simulator for equitable allocation of city resources. Algorithmic Fairness in Artificial intelligence, Machine learning and Decision making (FAIR-AMLD23), 2023.
- [50] Igor Nor, Danny Hermelin, Sylvain Charlat, Jan Engelstadter, Max Reuter, Olivier Duro, and Marie-France Sagot. Mod/resc parsimony inference: Theory and application. *Information and Computation*, 213:23–32, 2012.
- [51] James Orlin. Contentment in graph theory: covering graphs with cliques. In *Indagationes Mathematicae (Proceedings)*, volume 80, pages 406–424. Elsevier, 1977.
- [52] James B Orlin. Max flows in o (nm) time, or better. In *Proceedings of the forty-fifth annual ACM symposium on Theory of computing*, pages 765–774, 2013.
- [53] Zhe Pan, Xu Li, Shuibing He, Xuechen Zhang, Rui Wang, Yunjun Gao, Gang Chen, and Xian-He Sun. Ambea: Aggressive maximal biclique enumeration in large bipartite graph computing. *IEEE Transactions on Computers*, 73(12):2664–2677, 2024. doi: 10.1109/TC.2024.3441864.
- [54] Ryan A Rossi and Rong Zhou. Graphzip: a clique-based sparse graph compression method. *Journal of Big Data*, 5(1):10, 2018.
- [55] Hans Ulrich Simon. On approximate solutions for combinatorial optimization problems. *SIAM Journal on Discrete Mathematics*, 3(2):294–310, 1990.
- [56] Tamás G Tarjan. Complexity of lattice-configurations. 1975.
- [57] Zsolt Tuza. Covering of graphs by complete bipartite subgraphs; complexity of 0–1 matrices. *Combinatorica*, 4:111–116, 1984.
- [58] Jaideep Vaidya, Vijayalakshmi Atluri, Qi Guo, and Haibing Lu. Edge-rmp: Minimizing administrative assignments for role-based access control. *J. Comput. Secur.*, 17(2):211–235, April 2009.
- [59] Jaideep Vaidya, Vijayalakshmi Atluri, and Janice Warner. Roleminer: mining roles using subset enumeration. In *Proceedings of the 13th ACM Conference on Computer and Communications Security, CCS ’06*, page 144–153, New York, NY, USA, 2006. Association for Computing Machinery. doi: 10.1145/1180405.1180424.
- [60] Kevin Verbeek and Subhash Suri. Metric embedding, hyperbolic space, and social networks. In *Proceedings of the thirtieth annual symposium on Computational geometry*, pages 501–510, 2014.
- [61] Barry Wellman et al. Structural analysis: From method and metaphor to theory and substance. *Contemporary Studies in Sociology*, 15:19–61, 1997.

- [62] Jaewon Yang and Jure Leskovec. Defining and evaluating network communities based on ground-truth. In *Proceedings of the ACM SIGKDD workshop on mining data semantics*, pages 1–8, 2012.
- [63] I-Cheng Yeh. Default of Credit Card Clients. UCI Machine Learning Repository, 2009. DOI: <https://doi.org/10.24432/C55S3H>.
- [64] Xiaohan Zhao, Alessandra Sala, Haitao Zheng, and Ben Y Zhao. Efficient shortest paths on massive social graphs. In *7th International Conference on Collaborative Computing: Networking, Applications and Worksharing (CollaborateCom)*, pages 77–86. IEEE, 2011.
- [65] Xiaohan Zhao and Haitao Zheng. Orion: shortest path estimation for large social graphs. In *3rd Workshop on Online Social Networks (WOSN 2010)*, 2010.
- [66] David Zuckerman. Linear degree extractors and the inapproximability of max clique and chromatic number. In *Proceedings of the thirty-eighth annual ACM symposium on Theory of computing*, pages 681–690, 2006.

A Missing proofs from Section 2

PROOF OF LEMMA 2.1. We begin by showing the first direction. Let $\mathcal{S} = \{S_1, \dots, S_r\}$ be a biclique edge cover of G satisfying $\mu(\mathcal{S}) = \sum_{j=1}^r |\mathcal{D}(S_j)| \leq \tau$. We construct a cover $\mathcal{S}'$ for kG by expanding each biclique in $\mathcal{S}$. For each $S_j \in \mathcal{S}$, we define a corresponding subgraph S_j^k in kG such that $\mathcal{D}(S_j^k) = \{v_i \mid v \in \mathcal{D}(S_j), i \in [k]\}$. Since S_j is a biclique in G , it follows from the construction of kG that for every $v \in \mathcal{V}(S_j)$ and $u \in \mathcal{U}(S_j)$, all edges between copies of u and copies of v exist in kG . Therefore, S_j^k is a biclique in kG .

The collection $\mathcal{S}' = \{S_1^k, \dots, S_r^k\}$ forms a valid biclique edge cover for kG since every edge (u_i, v_j) in $\mathcal{E}(kG)$ corresponds to an edge (u, v) in $\mathcal{E}(G)$, which is covered by some S_l . Therefore, the edge (u_i, v_j) is covered by $S_l^k \in \mathcal{S}'$. The size of each blown-up biclique is $|\mathcal{D}(S_j^k)| = k \cdot |\mathcal{D}(S_j)|$, by definition. Therefore, the total weight portion of the cost of the cover $\mathcal{S}'$ is $\sum_{j=1}^r |\mathcal{D}(S_j^k)| = k \cdot \mu(\mathcal{S}) \leq k\tau$. Regarding the constant cost component of σ , note that $|\mathcal{S}'| = |\mathcal{S}| = r$. Since a trivial edge cover consisting of individual edges has size m (since $|E| = m$), we can assume without loss of generality that $r \leq m$. Thus, the total cost for $\mathcal{S}'$ is:

$$\sigma(\mathcal{S}') = c|\mathcal{S}'| + \sum_{j=1}^r |\mathcal{D}(S_j^k)| \leq cm + k\tau.$$

This completes the first part of the lemma.

Next, we show the other direction. Let $\mathcal{S}'$ be a biclique edge cover of kG satisfying the cost constraint. For all vertices $x \in \mathcal{D}(kG)$ let $n_x(\mathcal{S}')$ denote the number of bicliques in $\mathcal{S}'$ that contain the vertex x . The total weight component of the cost function $\sigma(\mathcal{S}')$ can be written as $\sum_{S' \in \mathcal{S}'} |\mathcal{D}(S')| = \sum_{x \in \mathcal{D}(kG)} n_x(\mathcal{S}')$. For each vertex $x \in \mathcal{D}(G)$, we identify the copy in kG that contributes the least to the weight of the cover. Formally, we define $i(x) = \arg \min_{j \in [k]} \{n_{x_j}(\mathcal{S}')\}$ to be the index of the copy of x appearing in the fewest bicliques in $\mathcal{S}'$. We construct a cover $\mathcal{S}$ for G by projecting the bicliques of $\mathcal{S}'$ onto these specific copies. Specifically, for each $S' \in \mathcal{S}'$, we construct a set $\pi(S') = \{x \in \mathcal{D}(G) \mid x_{i(x)} \in \mathcal{D}(S')\}$. We discard any empty sets resulting from this projection. Finally, we set $\mathcal{S} = \{\pi(S') \mid S' \in \mathcal{S}'\}$.

We first verify that $\mathcal{S}$ is a valid cover for G . Consider any edge $\{u, v\} \in E$. By the definition of kG , the copies $u_{i(u)}$ and $v_{i(v)}$ are connected by an edge in kG . Therefore, there must be some biclique $S' \in \mathcal{S}'$ covering the edge $\{u_{i(u)}, v_{i(v)}\}$. By our construction, u and v will both be included in $\pi(S')$, implying $\pi(S')$ covers the edge $\{u, v\}$. Furthermore, it is easy to check that for all $S' \in \mathcal{S}'$, the subgraph $\pi(S')$ is a biclique in G , since S' is a biclique in kG , by

definition. Thus, every $S \in \mathcal{S}$ is a biclique, and $\mathcal{S}$ is a valid biclique cover for G .

Now we analyze the cost $\mu(\mathcal{S})$. By definition, for all $x \in \mathcal{D}(G)$, the number of bicliques in $\mathcal{S}$ containing x (i.e., $n_x(\mathcal{S})$) is equal to $n_{x_{i(x)}}(\mathcal{S}')$. By our choice of $i(x)$, we have $n_{x_{i(x)}}(\mathcal{S}') \leq \frac{1}{k} \sum_{j=1}^k n_{x_j}(\mathcal{S}')$. Summing over all vertices in G , we obtain:

$$\begin{aligned} \mu(\mathcal{S}) &= \sum_{x \in \mathcal{D}(G)} n_{x_{i(x)}}(\mathcal{S}') \leq \sum_{x \in \mathcal{D}(G)} \frac{1}{k} \sum_{j=1}^k n_{x_j}(\mathcal{S}') \\ &= \frac{1}{k} \sum_{x \in \mathcal{D}(kG)} n_x(\mathcal{S}') \leq \frac{1}{k} \left(\sum_{x \in \mathcal{D}(kG)} n_x(\mathcal{S}') + c|\mathcal{S}'| \right) \\ &= \frac{1}{k} \sigma(\mathcal{S}') \leq \frac{1}{k} (k\tau + cm) = \tau + \frac{cm}{k}. \end{aligned}$$

Since $\mu(\mathcal{S})$ must be an integer (as it is a sum of vertex counts) and we chose $k > cm$, we have $\frac{cm}{k} < 1$. This implies $\mu(\mathcal{S}) \leq \tau$, and the result follows. $\square$

B Missing details from Section 3

B.1 Algorithm for the c -densest subgraph problem

We first review the exact algorithm for the densest subgraph problem. Given G , Goldberg [31] created a max-flow instance to check whether there exists a subgraph X such that $\frac{|\mathcal{E}(X)|}{|\mathcal{D}(X)|} \geq \gamma$, for any real number γ . They construct the directed graph $G'(V', E')$ such that $V' = V \cup U \cup \{s, t\}$, where s is the source and t is the sink node. Furthermore $E' = E \cup (\bigcup_{v \in V} (s, v)) \cup (\bigcup_{u \in U} (s, u)) \cup (\bigcup_{v \in V} (v, t)) \cup (\bigcup_{u \in U} (u, t))$. The direction of every edge in E is from a node in V to U and the capacity is 1. The capacity of each edge (s, v) (or (s, u)) is $\deg(v)$ (or $\deg(u)$), and the capacity of each edge (v, t) (or (u, t)) is 2γ , where $\deg(v)$ is the degree of node v in graph G . Let $\bar{X}$ be the subgraph of G such that $\mathcal{D}(\bar{X}) = (V \cup U) \setminus \mathcal{D}(X)$ and $\mathcal{E}(\bar{X}) = \{(u, v) \in E \mid v \in \mathcal{V}(\bar{X}), u \in \mathcal{U}(\bar{X})\}$. Let also $\mathcal{E}(X, \bar{X})$ be the edges in E between nodes in $\mathcal{D}(X)$ and $\mathcal{D}(\bar{X})$. They show that i) $\frac{|\mathcal{E}(X)|}{|\mathcal{D}(X)|} \geq \gamma \Leftrightarrow \sum_{v \in \mathcal{D}(\bar{X})} \deg(v) + |\mathcal{E}(X, \bar{X})| + 2\gamma|\mathcal{D}(X)| \leq 2|E|$, and ii) the cost of the s - t cut of G' defined by the nodes $\mathcal{D}(X)$ in the s side and nodes $\mathcal{D}(\bar{X})$ in the t side is exactly $\sum_{v \in \mathcal{D}(\bar{X})} \deg(v) + |\mathcal{E}(X, \bar{X})| + 2\gamma|\mathcal{D}(X)|$. Hence by the max-flow min-cut theorem, they compute a max-flow on G' and if its value is at most $2|E|$ and $X \neq \emptyset$ then they run their procedure for a larger value of γ . Otherwise, they run their procedure for smaller values of γ . By running a binary search on the values of γ they get the densest subgraph from G , which is the subgraph X returned in the last step of the binary search where the cost of the s - t min-cut was at most $2|E|$ and $X \neq \emptyset$.

For the c -densest subgraph problem, we have a similar construction. In fact, the graph G' we compute is exactly the same. We decide whether there exists a subgraph X such that $\frac{|\mathcal{E}(X)|}{|\mathcal{D}(X)|+c} \geq \gamma$. Following the analysis in [30, 31] we get $\frac{|\mathcal{E}(X)|}{|\mathcal{D}(X)|+c} \geq \gamma \Leftrightarrow \sum_{v \in \mathcal{D}(\bar{X})} \deg(v) + |\mathcal{E}(X, \bar{X})| + 2\gamma|\mathcal{D}(X)| \leq 2|E| - 2\gamma \cdot c$. We compute a max-flow on G' and if its value is at most $2|E| - 2\gamma \cdot c$ then we run our procedure for a larger value of γ . Otherwise, we run our procedure for smaller values of γ . We note that we can also run a

binary search on γ since the function

$$f(\gamma) = \min_{X \subseteq G} \left(\sum_{v \in \mathcal{D}(\bar{X})} \deg(v) + |\mathcal{E}(X, \bar{X})| + 2\gamma |\mathcal{D}(X)| \right) - 2|E| + 2\gamma \cdot c$$

is a non-decreasing function.

Overall, notice that γ can take $O(|E| \cdot (|V| + |U|))$ different values so the binary search executes $O(\log(\max\{|V|, |U|\}))$ steps. In each step of the binary search a maximum flow is computed on G' that contains $O(|V| + |U|)$ nodes and $O(|E|)$ edges. Using [52], the max-flow (min s - t cut) on G' is computed in $O((|V| + |U|)|E|)$ time. Using the recent breakthrough of the max-flow problem [18], we can compute the max-flow in each iteration of the binary search in $|E|^{1+o(1)}$ time.

Putting everything together, we conclude with Lemma 3.1.

B.2 Extension to other graph structures

For intersection bipartite graphs we extend the proof of Lemma 3.2. In the i -th iteration of the greedy procedure, let $\rho = \bigcap_{u \in \mathcal{U}(S^*)} r_u$, which is a hyper-rectangle in $\mathbb{R}^d$. We show that there exists a hyper-rectangle y in $\mathbb{R}^{2d}$ such that

$$\{v \in V \mid r_v \cap \rho \neq \emptyset\} = \{v \in V \mid p_v \in y\}. \quad (4)$$

We define the hyper-rectangle y in $\mathbb{R}^{2d}$ as follows,

$$y = (-\infty, \rho^{(1)+}] \times (-\infty, \rho^{(2)+}] \times \dots \times (-\infty, \rho^{(d)+}] \times [\rho^{(1)-}, \infty) \times [\rho^{(2)-}, \infty) \times \dots \times [\rho^{(d)-}, \infty).$$

Let $v \in V$ such that $r_v \cap \rho \neq \emptyset$. Two hyper-rectangles r_v, ρ intersect if and only if for each coordinate $j \in [d]$, their j -th interval intersects, i.e., the interval $[r_v^{(j)-}, r_v^{(j)+}]$ has a non-empty intersection with the interval $[\rho^{(j)-}, \rho^{(j)+}]$. Since these intervals intersect it holds that $r_v^{(j)-} \leq \rho^{(j)+}$ and $r_v^{(j)+} \geq \rho^{(j)-}$. Hence we notice that

$$r_v^{(1)-} \in (-\infty, \rho^{(1)+}], \dots, r_v^{(d)-} \in (-\infty, \rho^{(d)+}], r_v^{(1)+} \in [\rho^{(1)-}, \infty), \dots, r_v^{(d)+} \in [\rho^{(d)-}, \infty),$$

or equivalently $p_v \in y$. The same proof works in the opposite direction, so Equation (4) holds. Since there exists such hyper-rectangle y in $\mathbb{R}^{2d}$ all steps of the proof of Lemma 3.2 follow almost verbatim having $O(\log^{2d} n_V)$ canonical hyper-rectangles instead of $O(\log^d n_V)$.

Interval bipartite graphs. A graph $G(V, U, E, \mathcal{I}_V, \mathcal{I}_U)$ is an interval bipartite graph if for every $v \in V$ (resp. $u \in U$), v is associated with an interval $I_v = [I_v^-, I_v^+] \in \mathcal{I}_V$ (resp. u is associated with $I_u = [I_u^-, I_u^+] \in \mathcal{I}_U$) and $(v, u) \in E$ if and only if $I_v \cap I_u \neq \emptyset$. Notice that this is an intersection bipartite graph in ℓ_∞^1 . Hence, the next theorem follows directly from Theorem 4.1.

THEOREM B.1. *Given an interval bipartite graph $G(V, U, E, \mathcal{I}_V, \mathcal{I}_U)$ such that $n_V = |V|$, $n_U = |U|$ (and without loss of generality $n_V \leq n_U$), and $|E| = m$, there exists an $O(\log(n_U) \cdot \log^2(n_V))$ -approximation algorithm for the GWBC (and WBC) problem for any $c \geq 0$, that runs in $O(n_V \cdot (n_U + m) \cdot m \cdot \log(n_V))$ time.*

k -NN bipartite graphs. A graph $G(V, U, E, k)$ is a k -NN bipartite graph in $\mathbb{R}^d$ if $(v, u) \in E$ if and only if u is one of the k -nearest neighbors in U of v and v is one of the k -nearest neighbors in V of u . For every $u \in U$, we define the box r_u in $\mathbb{R}^d$ with side-length

$2\psi_\infty(u, k\text{-NN}(u, V))$, where $k\text{-NN}(u, V)$ is the k -nearest neighbor of u in V with respect to ℓ_∞^d . Let $\mathcal{R}_U = \{r_u \mid u \in U\}$ and let $r_V = V$, i.e., a set of zero volume hyper-rectangles. The graph $G'(V, U, E, \mathcal{R}_V, \mathcal{R}_U)$ is an intersection bipartite graph in $\mathbb{R}^d$. We note that the GWBC problem on G is not equivalent to the GWBC problem on G' because we did not consider the k nearest neighbors of V to U . However, we can still use the algorithm from Section 3.2.3 for the GWBC problem on G' with the one difference: for each canonical hyper-rectangle x , $E_x = \{(v, u) \mid v \in V_x, u \in U_x, \psi_\infty(v, u) \leq \psi_\infty(v, k\text{-NN}(v, U)), B(v, u) = 1\}$. We conclude with the next theorem.

THEOREM B.2. *Given a k -NN bipartite graph $G(V, U, E, k)$ in $\mathbb{R}^d$, where $d = O(1)$, such that $n_V = |V|$, $n_U = |U|$ (and without loss of generality $n_V \leq n_U$), and $|E| = m$, there exists an $O(\log(n_U) \cdot \log^d(n_V))$ -approximation algorithm for the GWBC (and WBC) problem for any $c \geq 0$, that runs in $O(n_V \cdot (n_U + m) \cdot m \cdot \log^{d-1}(n_V))$ time.*

B.3 Extension to any ℓ_α^d metric: Theoretical approach

We first define the notion of an ε -biclique. Given a graph G and an arbitrarily small constant $\varepsilon \in (0, 1)$, let $G'(V', U', E')$ be a graph where $V' = V$, $U' = U$, and $E' = \{(v, u) \mid \psi_\alpha(v, u) \leq (1 + \varepsilon)\delta\}$. Note that E' contains all edges of G , as well as additional pairs (v, u) with distance greater than δ but at most $(1 + \varepsilon)\delta$. An ε -biclique S' of G is defined as any biclique of the graph G' . Observe that every biclique of G is also an ε -biclique. An ε -biclique edge cover $\mathcal{S}$ is a collection of ε -bicliques whose union covers all edges of G .

Instead of a range tree, we use the construction and properties of the BBD-tree [6, 7]. A BBD-tree is a hierarchical geometric data structure designed for range aggregation queries in $\mathbb{R}^d$, where $d = O(1)$. Let $\mathcal{T}$ denote the BBD-tree constructed over V . The BBD-tree induces a collection $\mathcal{C}$ of $O(|V| \log |V|)$ canonical axis-aligned hyper-rectangles.¹ For any convex region $\rho \subseteq \mathbb{R}^d$, there exists a subset $C(\rho) \subseteq \mathcal{C}$ of $O(\varepsilon^{-d} \log |V|)$ pairwise disjoint hyper-rectangles such that $V \cap \rho \subseteq \bigcup_{x \in C(\rho)} (V \cap x) \subseteq V \cap (\oplus_\varepsilon \rho)$. Here, $\oplus_\varepsilon \rho$ denotes the Minkowski sum of ρ with a ball of radius $\varepsilon \cdot \text{diam}(\rho)$, that is, the region obtained by expanding ρ by $\varepsilon \cdot \text{diam}(\rho)$ in all directions, where $\text{diam}(\rho)$ denotes the diameter of ρ . As in the range tree construction, we assume without loss of generality that each hyper-rectangle $x \in \mathcal{C}$ is minimal with respect to V .

The algorithm for general ℓ_α^d metrics closely follows the algorithm from Theorem 3.4, with the following modifications: (i) the hash table B is defined over all pairs (v, u) with $v \in V$ and $u \in U$; (ii) we use a BBD-tree $\mathcal{T}'$ to get $\mathcal{C}$ constructed over V ; and (iii) in each iteration, for every hyper-rectangle $x \in \mathcal{C}$, we define a graph $G_x(V_x, U_x, E_x)$ where $V_x = V \cap x$, $U_x = \{u \in U \mid x \subseteq \oplus_\varepsilon \rho_u\}$, and $E_x = \{(v, u) \mid v \in V_x, u \in U_x, \psi_\alpha(v, u) \leq (1 + \varepsilon)\delta, B(v, u) = 1\}$.

Using the same arguments as in the correctness proof of Lemma 3.2, while accounting for the multiplicative $(1 + \varepsilon)$ approximation and the covering guarantees of the BBD-tree, it is straightforward to

¹A hyper-rectangle in the canonical set of a BBD-tree may contain a hole; that is, each region $x \in \mathcal{C}$ can be represented as the difference of two hyper-rectangles. For simplicity, we treat each such region x as a standard hyper-rectangle, since all properties and arguments we claim remain unchanged.

show that the algorithm returns an ε -biclique edge cover $\mathcal{S}$ satisfying $\sigma(\mathcal{S}) \leq O((\varepsilon^{-d} + \log n_V)n_U) \cdot \sigma(\mathcal{S}^*)$, where $\mathcal{S}^*$ denotes an optimal biclique edge cover of G . We emphasize that $\mathcal{S}^*$ is not an optimal ε -biclique cover. Skipping the remaining details, we conclude with the following theorem.

THEOREM B.3. *Given a δ -disk bipartite graph $G(V, U, E)$ in ℓ_α^d , where $d = O(1)$ and $\alpha \geq 1$, $n_V = |V|$, $n_U = |U|$ (with $n_V \leq n_U$),*

and $|E| = m$, there exists an algorithm that returns an ε -biclique edge cover $\mathcal{S}$ such that $\sigma(\mathcal{S}) \leq O((\varepsilon^{-d} + \log n_V) \log n_U) \cdot \sigma(\mathcal{S}^)$, where $\mathcal{S}^*$ is an optimal solution to the GWBC problem for any fixed parameter c . The algorithm runs in $O(n_V^3 n_U^3 \log n_V \log n_U)$ time and uses $O(n_U n_V + n_V \log n_V)$ space.*

C Additional experiments

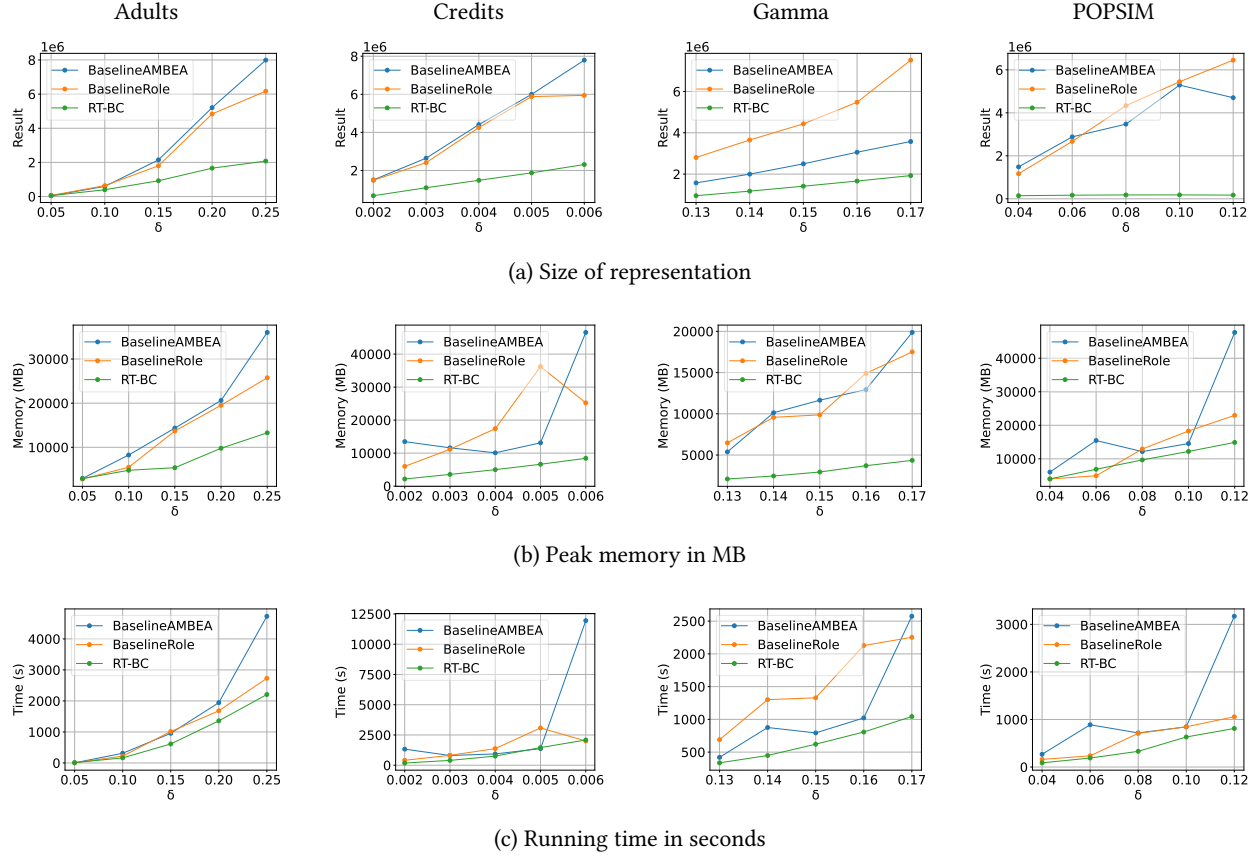


Figure 5: Experimental results for WBC in ℓ_∞^d across all datasets

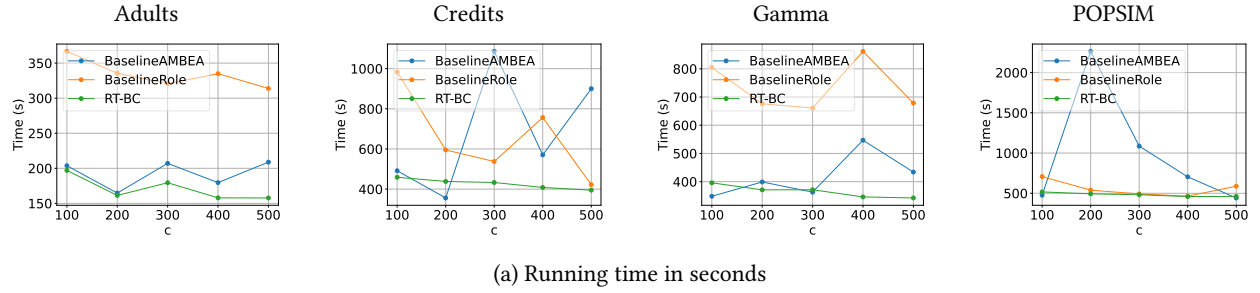


Figure 6: Experimental results for GWBC in ℓ_∞^d across all datasets

Dataset	δ	m	n	d	Grouping Attribute
Adults	0.05	64,280	14,846	5	Sex
	0.1	854,381			
	0.15	2,589,297			
	0.2	5,990,399			
	0.25	8,714,044			
Credit	0.002	1,538,057	28,726	6	Sex
	0.003	2,912,661			
	0.004	4,437,255			
	0.005	6,034,989			
Gamma	0.006	7,805,033	18,905	10	Class
	0.13	1,511,959			
	0.14	1,961,614			
	0.15	2,477,054			
	0.16	3,057,096			
POPSIM	0.17	3,702,052	8,591	2	Race
	0.04	1,753,508			
	0.06	3,090,592			
	0.08	4,328,986			
	0.1	5,442,100			
	0.12	6,449,440			

Table 3: Statistics of the δ -disk graphs used in our experiments for ℓ_∞^d .